



BIRZEIT UNIVERSITY

**FACULTY OF ENGINEERING AND TECHNOLOGY
DEPARTMENT OF ELECTRICAL AND COMPUTER
ENGINEERING**

ENEE5300 - Graduation Project

Smart Blind System Support

Prepared by:

Kareem Taha – 1211155

Meelad AbuAlzuluf – 1210009

Abdalrahman Shaheen – 1211753

Supervised by:

Dr. Wasel Ghanem

ENEE5300 – Section (5)

A Graduation Project submitted to the Department of Electrical and Computer Engineering
in partial fulfilment of the requirements for the degree of B.Sc. in Electrical and Computer
Engineering

July 2026

Abstract

Smart Blind System Support is a fully-offline, bilingual (English and Arabic) navigation aid that gives blind and visually-impaired pedestrians independent turn-by-turn guidance across the Birzeit University campus. Outdoors, a u-blox NEO-M9N GNSS receiver, measured at about 0.6 m at best and roughly 1 m on average on campus (several times tighter than a smartphone's GPS), is fused with BNO086 IMU dead-reckoning to hold position when satellite signals drop. Indoors, a network of Decawave DW1000 Ultra-Wideband anchors uses Two-Way Ranging and least-squares trilateration for roughly 50 cm operational accuracy. The indoor positioning was built and tested as a standalone setup on a local server, and the indoor-outdoor handoff at building entrances is designed but not yet integrated into the app. A self-built ESP32 wearable streams fused sensor data over Bluetooth Low Energy to a Flutter application that renders offline OpenStreetMap tiles through MapLibre, computes pedestrian routes with the embedded Valhalla engine, and speaks English or Arabic guidance through the phone's text-to-speech. The hardware runs on a custom two-layer PCB powered by a 2S lithium-ion pack for an estimated eight hours of use. The system was field-tested with blind students on campus.

المستخلص

نظام Smart Blind System Support هو نظام ملاحية يعمل دون اتصال بالإنترنت، ثنائي اللغة (العربية والإنجليزية)، يمنح المشاة المكفوفين وضعاف البصر إرشاداً مستقلاً خطوة بخطوة عبر حرم جامعة بيرزيت. في الخارج، تُدمج وحدة استقبال GNSS من نوع u-blox NEO-M9N، التي بلغت دقتها 0.7 م على أرض الحرم، أي أفضل بنحو ست مرات من نظام GPS المدمج في الهاتف، مع مستشعر القياس بالقصور الذاتي (IMU) من نوع BNO086 للحفاظ على الموقع بالحساب التتبعي عند انقطاع إشارات الأقمار. في الداخل، تستخدم شبكة من نقاط الإرساء من نوع Decawave DW1000 العاملة بالنطاق العريض فائق الضيق UWB قياس المدى ثنائي الاتجاه و Trilateration بطريقة Least Squares لتحقيق دقة تشغيلية تبلغ نحو 50 سم؛ إذ تغطي كل نقطة إرساء موجهة نحو الحيز ما يقارب نصف دائرة يمتد من 20 إلى 30 متراً أمامها، ويُحدد الموقع حيث تتداخل تغطية عدد كافٍ من نقاط الإرساء. وقد بُني نظام تحديد الموقع الداخلي واختبر كوحدة مستقلة تعمل على localhost، في حين أنّ الانتقال بين الوضعين الخارجي والداخلي عند مداخل المباني مُصمَّم إلا أنه لم يُدمج بعد في التطبيق. وحدة قابلة للارتداء مبنية ذاتياً على شريحة ESP32 تبث بيانات المستشعرات المدمجة عبر البلوتوث منخفض الطاقة إلى تطبيق Flutter يعرض بلاطات OpenStreetMap دون اتصال عبر MapLibre، ويحسب مسارات المشاة بمحرك التوجيه Valhalla المدمج، وينطق الإرشاد بالعربية أو الإنجليزية عبر نظام تحويل النص إلى كلام في الهاتف. تعمل المنظومة على لوحة دوائر مطبوعة مخصصة من طبقتين تُغذى ببطارية ليثيوم أيون ثنائية الخلية، بما يتيح وقت تشغيل يُقدَّر بثلاث ساعات. وقد جرى اختبار النظام ميدانياً مع طلاب مكفوفين في حرم جامعة بيرزيت.

Contents

Abstract	ii
Acronyms and Abbreviations	vi
List of Figures	ix
List of Tables	x
1 Introduction	1
1.1 Project Overview	1
1.2 Problem Statement	1
1.3 Motivation	2
1.4 Objectives	2
1.5 Report Organization	3
2 Background and Technology Selection	4
2.1 Localization and Navigation Overview	4
2.2 Outdoor Positioning	4
2.3 Indoor Positioning	5
2.3.1 WiFi RSSI	5
2.3.2 BLE Beacons	6
2.3.3 LoRa	7
2.3.4 Camera, LiDAR, and Magnetic Methods	7
2.3.5 Ultra-Wideband (Chosen)	7
2.4 UWB Ranging Principle	7
2.5 Embedded Platform and Connectivity	8
2.6 Mapping, Routing and Accessibility	9
2.7 Related Work	10
3 System Design and Architecture	12
3.1 System Overview	12
3.2 Hardware Architecture	13
3.3 Software Architecture	15

3.4	Indoor–Outdoor Handoff	17
3.5	Data Flow	18
4	Implementation	19
4.1	Outdoor Positioning	19
4.2	Testing WiFi RSSI	21
4.3	Indoor UWB Positioning	22
4.4	Routing and Offline Maps	24
4.5	Mobile Application	25
4.6	Accessibility and Guidance	26
4.7	Area-Aware Command Filtering	27
4.8	UWB Anchor Deployment	29
4.9	Carrier PCB and Power	30
5	Results, Testing and Evaluation	34
5.1	Outdoor GPS Accuracy	34
5.2	Evaluation of a Rejected Approach: WiFi RSSI Feasibility	36
5.3	Indoor UWB Accuracy	38
5.4	Power and Runtime	39
5.5	Field Testing	40
5.6	Delivered Feature Set and Limitations	42
6	Conclusions and Future Work	43
6.1	Conclusions	43
6.2	Future Work	44
A	Technical Specifications	A1
A.1	Wearable Module Pinout	A1
A.2	Serial Bus Comparison	A2
A.3	GNSS Receiver Selection: NEO-M9N versus NEO-M8N	A2
A.4	UWB Transceiver Selection: DW1000 versus DW3000	A3
B	Demonstration Videos	B1

Acronyms and Abbreviations

Acronym	Meaning
2D	Two-Dimensional
3D	Three-Dimensional
API	Application Programming Interface
BLE	Bluetooth Low Energy
BMS	Battery Management System
BNO086	CEVA/Hillcrest 9-axis smart IMU sensor (model number)
BVI	Blind and Visually Impaired
CC/CV	Constant Current / Constant Voltage (battery charging mode)
DOP	Dilution of Precision
DR	Dead Reckoning
DW1000	Decawave (Qorvo) UWB transceiver chip
EN	English
ESP32	Espressif Systems ESP32 microcontroller family
ESP32-S3	ESP32 variant with AI acceleration and full-speed USB OTG
Galileo	European GNSS constellation
GLONASS	Global Navigation Satellite System (Russian constellation)
GNSS	Global Navigation Satellite System
GPIO	General-Purpose Input/Output
GPS	Global Positioning System (United States GNSS)
HTTP	Hypertext Transfer Protocol
I ² C	Inter-Integrated Circuit (two-wire serial bus)
IEEE	Institute of Electrical and Electronics Engineers
IMU	Inertial Measurement Unit
JSON	JavaScript Object Notation

Acronym	Meaning
LiPo	Lithium Polymer (battery chemistry)
LOS	Line-of-Sight
M9N	Shorthand for the u-blox NEO-M9N GNSS module
MBTiles	MapBox Tiles (offline map container format)
MCU	Microcontroller Unit
MP1584	Monolithic Power Systems 3 A step-down buck converter
NEO-M8N	u-blox NEO-M8N concurrent GNSS receiver (earlier model)
NEO-M9N	u-blox NEO-M9N standard-precision GNSS module
NLOS	Non-Line-of-Sight
NMEA	National Marine Electronics Association GPS data protocol
OSM	OpenStreetMap
PCB	Printed Circuit Board
PDR	Pedestrian Dead Reckoning
RSSI	Received Signal Strength Indicator
RTL	Right-to-Left (text direction)
SBAS	Satellite-Based Augmentation System
SoC	System on Chip
SPI	Serial Peripheral Interface
ToF	Time-of-Flight
TTS	Text-to-Speech
TWR	Two-Way Ranging
UART	Universal Asynchronous Receiver-Transmitter
UWB	Ultra-Wideband
WiFi	Wireless Fidelity (IEEE 802.11)

List of Figures

2.1	Taxonomy of pedestrian localization technologies (adopted vs. rejected).	4
2.2	GPS positioning by satellite trilateration [1].	5
2.3	Illustrative log-distance path-loss for three clutter levels.	6
3.1	Three-tier system architecture.(Target design.)	12
3.2	Wearable hardware interconnect.	13
3.3	Power subsystem topology.	15
3.4	Software architecture.	16
3.5	Four-state indoor–outdoor handoff state machine.	17
3.6	End-to-end data flow.(Target design; the current build runs indoor trilateration on a separate server — see Chapter 5.)	18
4.1	Wearable bench build: the ESP32-S3 with the GNSS and IMU modules.	19
4.2	ESP32 scanning the campus WiFi access points during data collection.	21
4.3	UWB antenna-delay calibration setup.	23
4.4	Campus sidewalk and footpath network detail.	25
4.5	Mobile app home screen.	27
4.6	Offline voice destination search.	27
4.7	Arabic (right-to-left) settings screen.	28
4.8	Area-aware filtering of recovery prompts. The router remains the sole routing authority; the layer only suppresses a redundant cue when the user is provably still on a route-relevant walkable surface, and fails open otherwise.	29
4.9	UWB anchor on the 3D-printed ball-joint mount.	30
4.10	Mount assembly: ceiling clip and ball-joint base.	30
4.11	Deployed UWB anchor on its ceiling mount.	31
4.12	Annotated layout of the 2-layer carrier PCB.	32
4.13	Front and back of the fabricated carrier boards.	33
4.14	Bench power build.	33
5.1	In-app diagnostic card: phone vs. NEO-M9N.	35
5.2	A richly mapped city in a 3D map tool.	36
5.3	Birzeit campus in the same tool: flat and sparse.	36

5.4	WiFi RSSI positioning error vs. distance.	37
5.5	WiFi RSSI decay and fitted path-loss curve.	37
5.6	Measured vs. true UWB distance, 1–5 m.	39
5.7	Three-anchor trilateration layout (anchor and tag antenna-delay constants shown).	39
5.8	Blind student field-testing on a campus route.	41
5.9	Al-Sadik floor plan used for the UWB test.	41

List of Tables

2.1	ESP32 variants considered.	9
2.2	WiFi versus BLE as the data transport.	9
2.3	BLE 4.2 versus 5.0 for the wearable.	10
2.4	Indoor positioning techniques and representative accuracy [2].	11
3.1	Wearable components and interfaces.	14
3.2	Outdoor BLE packet and indoor serial telemetry.	18
4.1	Per-board UWB antenna-delay constants.	24
4.2	Off-route sensitivity levels in the guidance engine.	26
4.3	Power-source options considered for the wearable.	32
5.1	Paired phone and NEO-M9N fixes at the same points.	34
5.2	Phone GPS accuracy by handset.	35
5.3	Al-Sadik RSSI Data	37
5.4	UWB ranging error vs. true distance.	38
5.5	Per-subsystem current draw, theoretical vs. measured at 3.3 V. Share is each part's fraction of the 190 mA measured with all subsystems active.	40
5.6	Delivered feature set and its tested state.	42
A.1	ESP32 wearable pin assignment.	A1
A.2	Serial bus comparison.	A2
A.3	NEO-M9N versus NEO-M8N.	A3
A.4	DW1000 versus DW3000.	A3

Chapter 1

Introduction

1.1 Project Overview

Smart Blind System Support is a wearable navigation aid that gives blind and visually-impaired pedestrians turn-by-turn walking guidance across the Birzeit University (BZU) campus. Two constraints shaped every design decision: the system runs fully offline, and it speaks to the user in English or Arabic, switchable at any time.

It positions the user in two domains:

- **Outdoors:** a u-blox NEO-M9N multi-GNSS module (GPS, GLONASS, Galileo, Bei-Dou) is rated at 1.5 m CEP and measured about 0.6 m at best and roughly 1 m on average on campus, with a BNO086 IMU bridging brief signal gaps by dead-reckoning for up to 30 s.
- **Indoors:** a network of ESP32 UWB anchors (Qorvo DW1000) ranges to a wearable tag, and a separate local server trilaterates the position to about 50 cm. Each anchor is aimed into the space and covers roughly a half-circle out to 20–30 m in front of it (verified in real testing); the tag is located where enough anchor fields overlap. Indoor positioning was developed and tested as a standalone setup, not yet integrated into the phone app.

Guidance comes from the Valhalla offline routing engine running on OpenStreetMap campus tiles stored on the device. The wearable itself is a custom ESP32-S3 unit on a purpose-built carrier PCB that streams fused sensor data to the phone over Bluetooth Low Energy. The phone handles routing, speech, landmark alerts, and vibration feedback, and is designed to switch between outdoor and indoor guidance at building entrances (indoor mode is a standalone test setup in the current build).

1.2 Problem Statement

A visually-impaired student on the BZU campus faces problems that ordinary navigation apps do not solve:

- GPS degrades near building facades and fails indoors, exactly where guidance is most needed.

- Most assistive apps need a constant internet connection, which is unreliable on campus and unavailable during the power cuts common in the West Bank.
- Devices with Arabic audio are rare and priced well above what local students can afford.
- Sidewalks around campus are uneven or absent, so routing that ignores walkability and road crossings is a hazard, not an aid.

None of the assistive-navigation systems reviewed for this project combines offline operation, Arabic guidance, UWB-based indoor positioning, and affordable purpose-built hardware. The system was built to close that gap for the BZU campus.

1.3 Motivation

Independent mobility separates a student who must coordinate every trip with a guide from one who moves through campus freely; removing that dependency has a direct effect on daily autonomy. The need is large: the World Health Organization estimates that over 2.2 billion people live with vision impairment [3], and indoor navigation in particular remains largely unsolved for assistive technology.

The underlying technology is maturing quickly. The indoor-positioning market was valued at roughly USD 20.38 billion in 2023 and is projected to reach about USD 174.02 billion by 2030, a compound annual growth rate near 37.6% [4]. That investment is making the exact components this project relies on (UWB, BLE, and sensor fusion) cheaper and better documented. The system deliberately uses commodity ESP32 hardware, the open-source Flutter framework, and offline tiles so that another campus, in Palestine or elsewhere, could reproduce the system at low cost.

1.4 Objectives

- Sub-metre outdoor positioning from the NEO-M9N fused with IMU dead-reckoning (up to 30 s).
- Centimetre-scale indoor positioning from DW1000 UWB two-way ranging and least-squares trilateration, about 50 cm operational; each anchor covers roughly a half-circle out to 20–30 m, and the tag is placed where enough anchor fields overlap.
- Fully offline operation: routing tiles, speech models, the points-of-interest database, and all audio reside on the device.
- Bilingual English/Arabic guidance across voice search, navigation announcements, and the full UI, switchable at any time.

- A purpose-built wearable: a custom ESP32-S3 carrier PCB integrating the NEO-M9N, BNO086 IMU, and BLE radio, powered by a 2S lithium-ion pack with BMS protection and buck regulation.
- Automatic indoor-to-outdoor and outdoor-to-indoor handoff at building entrances, with no user action required.
- Area-aware command filtering that lets the user move freely across an allowed walkable area toward the destination instead of being pinned to the exact route line, suppressing redundant off-route prompts while they remain on a route-relevant surface.
- Vibration cues on confirmed turns, giving tactile confirmation in noisy surroundings.
- Safe street-crossing announcements at road junctions along campus routes.
- Nearby-facility announcements (name, side, and distance) for the 39 named points of interest in the campus database.
- Automatic fallback to the phone's GPS and motion sensors when the wearable module is not connected.

1.5 Report Organization

The rest of this report is organized as follows:

- **Chapter 2** reviews the background and the technology choices behind the system: the localization options considered, the mapping and routing approach, and the accessibility requirements.
- **Chapter 3** describes the system design and architecture: the hardware, the software, the indoor-to-outdoor handoff, and the data flow between components.
- **Chapter 4** covers the implementation: outdoor positioning, indoor UWB localization, routing and maps, the mobile application, accessibility features, area-aware command filtering, anchor deployment, and the carrier PCB and power subsystem.
- **Chapter 5** reports the results, testing, and evaluation, including GPS and UWB accuracy, the rejected WiFi-RSSI study, power measurements, and field testing with users.
- **Chapter 6** presents the conclusions and future work.

Chapter 2

Background and Technology Selection

The system uses two positioning technologies, one outdoors and one indoors, because no single method handles both well. This chapter records the choices behind that split: for each part of the problem it lists the options, names the one kept, and gives the reason.

2.1 Localization and Navigation Overview

Knowing where the user stands is the central difficulty, and the answer divides by setting. Outdoors, satellite navigation is available and accurate; inside a building those signals collapse, and every replacement trades some mix of accuracy, cost, and added infrastructure. Fig. 2.1 lays out the candidates weighed: green nodes were adopted, grey ones studied and dropped.

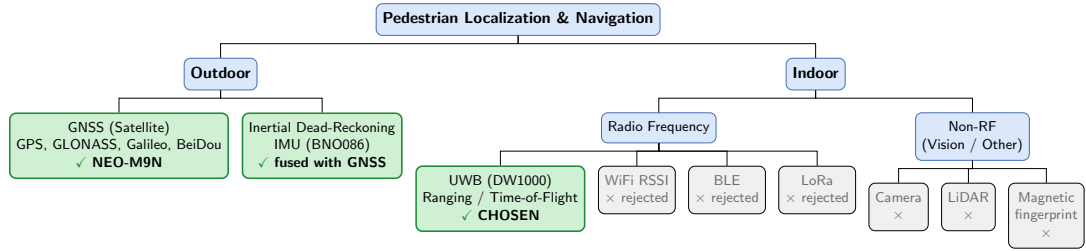


Fig. 2.1: Taxonomy of pedestrian localization technologies (adopted vs. rejected).

Outdoors the design keeps multi-constellation GNSS and fuses it with an inertial measurement unit for short dead-reckoning gaps (Section 2.2). Indoors the choice is ultra-wideband ranging (Sections 2.3 and 2.4). The grey nodes cover the rest of the tree: on the radio side WiFi received-signal-strength, BLE beacons, and LoRa were ruled out for positioning, and on the non-radio side camera, LiDAR, and magnetic-field methods too. All five rejections are argued in Section 2.3. WiFi and BLE return in Section 2.5, but only as ways to carry data to the phone, a separate question from positioning.

2.2 Outdoor Positioning

GNSS. Four constellations are visible over the campus: GPS, GLONASS, Galileo, and BeiDou [5]. A receiver that locks onto all four keeps far more satellites in view than a single-system unit, lowering the geometric dilution of precision and holding the fix where a building wall would block half the sky. Satellite-based augmentation (SBAS) adds wide-area corrections toward sub-metre error with no local base station. Solving for position plus the

receiver clock needs at least four satellites, and each extra one tightens the estimate, the geometry sketched in Fig. 2.2.

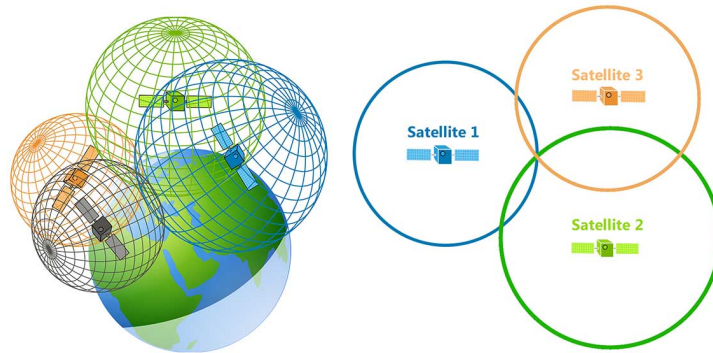


Fig. 2.2: GPS positioning by satellite trilateration [1].

Choosing the receiver. An early prototype carried a u-blox NEO-M8N, which tracks only GPS and GLONASS and settled at roughly 1.2 to 2.5 m of horizontal error under a partly blocked sky [6]. Pedestrian guidance fires turn cues around 20 m out, so that error band was too wide. The NEO-M9N replaced it [5]: it tracks all four constellations plus SBAS, streams a fix at up to 10 Hz over a 38 400-baud UART, and cold-starts in under 30 s. Its datasheet rates the horizontal accuracy at 1.5 m CEP [7]; in repeated same-point campus trials it did better, settling to about 0.6 m at best and roughly 1 m on average, well inside the phone GPS logged beside it. Every outdoor figure in this report comes from the M9N.

Heading and dead-reckoning. A BNO086 nine-axis sensor reports a fused orientation quaternion over I²C, so the microcontroller runs no filtering code of its own. That heading feeds both the turn logic and the fallback for when satellites drop out: if the fix degrades past 20 m of uncertainty for two seconds, the system stops trusting it and dead-reckons from the last good position using the IMU heading and a fixed stride length, capped at 30 s, enough to bridge a doorway or short covered stretch. The fix and the heading ride as separate BLE-packet fields and the phone decides which to believe; no Kalman filter runs on the wearable.

2.3 Indoor Positioning

Once the user steps inside, concrete and steel absorb most of the satellite energy and reflect the rest, so GNSS is not an option. Five non-UWB methods were considered as replacements, and each was set aside for a concrete reason.

2.3.1 WiFi RSSI

WiFi received-signal-strength looks attractive because it leans on access points the university already runs, at no extra hardware cost. Two flavours were tried. The model-based one turns

signal strength into a distance through the log-distance path-loss relation:

$$\text{RSSI}(d) = A - 10n \log_{10}\left(\frac{d}{d_0}\right) \quad (2.1)$$

The model hinges on the path-loss exponent n , near 2 in free space but rising to 2.5–4.0 once walls scatter the signal. Distances from three access points then fix a position by trilateration. The second flavour, fingerprinting, skips the model and surveys the signal pattern at a grid of points beforehand, matching each live reading against that stored map.

That exponent is what makes the model fragile. As shown in Fig. 2.3, raising n from 2.0 toward 4.0 reshapes the curve, so no single indoor value holds and the same drop in strength maps to a different distance depending on the walls and furniture in the path. The curves plot the theoretical model with example reference values; the system’s own campus feasibility measurements are in Section 5.2.

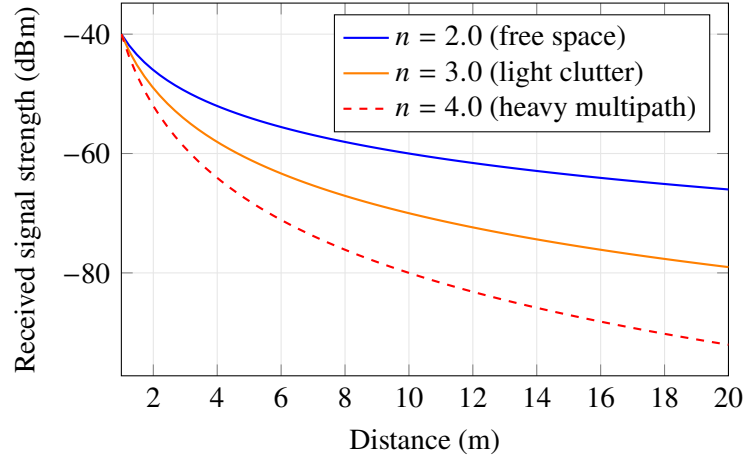


Fig. 2.3: Illustrative log-distance path-loss for three clutter levels.

Multipath is the root cause: reflections off walls, floors, and people swing the reading from moment to moment even when the receiver is still. WiFi RSSI therefore fails on three counts: indoor strength varies too much to hold a steady sub-3 m fix; a relocated, upgraded, or rebooted access point quietly invalidates an offline fingerprint map; and keeping a multi-floor fingerprint database current after deployment is unsustainable. That reliability is unsafe for spoken turn cues.

2.3.2 BLE Beacons

BLE beacons read distance from advertised signal strength with the same path-loss idea, and the literature puts them at roughly 1 to 3 m indoors [8]. They inherit WiFi’s fluctuation problem unchanged. Reaching sub-metre accuracy would need beacons at a density near the

UWB anchor count without the matching accuracy, and beacon scanning would share the very radio the wearable uses for its data link. BLE beacons were dropped.

2.3.3 LoRa

LoRa trades data rate for kilometre-scale range and is positioned by time-difference of arrival across synchronised gateways. None of that fits: the range is wasted on a campus about 450 m across, the method demands gateway clocks aligned to within tens of nanoseconds, and its reported indoor accuracy of 1 to 5 m is far too coarse for turn cues [9].

2.3.4 Camera, LiDAR, and Magnetic Methods

Camera and LiDAR localization can reach under 10 cm against a prebuilt visual or point-cloud map [10], and magnetic positioning exploits the stable field distortions a steel-framed building creates [11]. All three were excluded on practical grounds. Body-worn cameras and LiDAR push cost, size, and weight past what a discreet wearable allows and raise a privacy objection for a device pointed at everyone nearby. Magnetic methods need a full building survey that must be redone after every renovation, which is frequent here.

2.3.5 Ultra-Wideband (Chosen)

UWB cleared the bar the others tripped over. It measures range from pulse timing rather than signal strength, so the fluctuation that sank the radio alternatives never enters the calculation. Reported accuracy runs from 0.05 to 0.3 m, one to two orders of magnitude better than WiFi or BLE strength methods [12, 13], and a handful of small, inexpensive DW1000 anchors cover a space: each anchor, aimed into the area, covers roughly a half-circle out to 20–30 m in front of it, and the tag is located wherever enough of these fields overlap. The next section explains how the ranging works.

2.4 UWB Ranging Principle

The signal. The Qorvo DW1000 sends sub-nanosecond pulses spread across several gigahertz of bandwidth [14]; the library’s `LONGDATA_RANGE_LOWPOWER` mode used here places that on Channel 5 (near 6.5 GHz) with a 16 MHz pulse-repetition frequency [15]. That wide bandwidth is the source of the precision: a one-nanosecond pulse covers only about 30 cm of travel, so the receiver can pick out the first straight-line arrival and ignore the later reflections behind it. The multipath that defeats strength-based methods becomes a non-issue.

Two-way ranging. Anchor and tag measure range by two-way ranging (TWR): the tag polls, the anchor replies after a fixed known interval, and the tag converts the round trip into a range. Because that turnaround delay is a known constant, clock drift between the two radios cancels out, so the nodes need no synchronisation. Time-difference of arrival would require exactly that synchronisation, one more reason it was avoided. In firmware the setup is:

- the open-source arduino-dw1000 library [16], in `MODE_LONGDATA_RANGE_LOWPOWER`, which favours range and noise immunity over peak data rate;
- three anchors at fixed addresses `0xA1`, `0xA2`, and `0xA3`, each polled in turn at roughly 10 Hz; and
- a per-board antenna-delay calibration, with the values and the procedure given in Chapter 4.

Trilateration. UWB fixes position by the same trilateration principle GPS uses, three ranges intersecting (Fig. 2.2), so no separate diagram is needed here. Three ranges to anchors at known coordinates define three circles. Measurement noise means they seldom meet at one point, so the position is solved in the least-squares sense at the spot that best fits all three. The solve runs on the edge rather than in the app; where it sits and how the result reaches the phone are covered in Chapter 4.

2.5 Embedded Platform and Connectivity

The processor. The wearable must run a BLE stack, drive the DW1000 over SPI, read the BNO086 over I²C and the NEO-M9N over UART, and manage its own power, all in a small, low-power package. Three platforms were weighed against that list:

- **Arduino (AVR):** no radio on board and too little flash for a BLE stack, on an 8-bit core that cannot push quaternion math through inside a 100 ms loop.
- **Raspberry Pi:** spends 20 to 30 s booting an operating system, idles in the hundreds of milliwatts, and carries a CPU, DRAM, and storage a sensor hub has no use for, all at several times the price.
- **ESP32-S3 (chosen):** integrated WiFi and BLE 5.0, around 45 GPIO, up to 8 MB of PSRAM, and a dual-core LX7 running FreeRTOS with sub-millisecond scheduling, plus ready library support for both the DW1000 and the BNO086 [17].

An STM32-class MCU was also considered: it offers strong real-time performance, but has no integrated wireless, so it would need an external BLE/UWB radio and the extra board complexity that brings. The ESP32-S3's on-chip BLE 5.0 and ready library support kept the design simpler, which is why the choice stayed inside the ESP32 family. Table 2.1 sets that family side by side and shows why the S3 won.

Transport: WiFi versus BLE. The radios turned down for positioning return here for a different job: shuttling sensor readings to the phone. Accuracy is beside the point; what matters is power, pairing, and infrastructure. WiFi needs a router or hotspot, draws more

Table 2.1: ESP32 variants considered.

Variant	Cores	Wireless	GPIO	PSRAM	Notes
ESP32	2× LX6	WiFi + BT 4.2	34	–	Original part; no BLE 5.0, no PSRAM
ESP32-S2	1× LX7	WiFi only	43	2 MB	No Bluetooth at all; unusable here
ESP32-S3	2× LX7	WiFi + BLE 5.0	~45	8 MB	Selected: dual-core, full BLE 5.0, 8 MB PSRAM
ESP32-C3	1× RISC-V	WiFi + BLE 5.0	22	–	Single-core; too few GPIO

current, and occupies the phone’s wireless interface. BLE bonds once, reconnects on its own, stays peer-to-peer, and leaves the phone’s internet untouched. The sensor packet also fits inside one BLE notification, so no fragmentation is needed. Table 2.2 compares the two.

Table 2.2: WiFi versus BLE as the data transport.

Property	WiFi	BLE
Power draw	High; continuous RF	Low; duty-cycled
Infrastructure	Needs router or hotspot	None; direct peer-to-peer
Phone internet	Shares the phone’s connection	Leaves it free
Pairing	SSID + password each session	One-time bond, auto-reconnect
Fit for the packet	Overkill for the sensor packet	Fits in one notification

Within BLE the version matters because of the advertising payload limit. BLE 4.2 caps a standard advertisement at 31 bytes; BLE 5.0 extended advertising raises that to 255 [18]. The packet exceeds the 31-byte BLE 4.2 advertising limit, so BLE 5.0 extended advertising is required, making the ESP32-S3’s on-chip BLE 5.0 a hard requirement. Table 2.3 lists the relevant differences. Pairing is a single six-digit PIN entered once; after that the link reconnects with no Bluetooth menu, essential for a user who cannot work a visual pairing dialog.

Here WiFi and BLE compete as *transport* and BLE wins; as *positioning* methods in Section 2.3 both lost to UWB.

2.6 Mapping, Routing and Accessibility

Map data. OpenStreetMap (OSM) can be pulled down once and used entirely offline [19]. Google Maps cannot: its terms forbid caching tiles offline and it expects a live connection at query time, which rules it out for a device meant to work with no signal. OSM also

Table 2.3: BLE 4.2 versus 5.0 for the wearable.

Feature	BLE 4.2	BLE 5.0
Advertising payload	31 bytes (standard)	255 bytes (extended)
Maximum data rate	1 Mb/s	2 Mb/s (LE 2M PHY)
Extended range mode	Not available	4× via LE Coded PHY
Security	AES-CCM; LE legacy pairing	AES-CCM + LE Secure Connections

carries the pedestrian tags the router needs: `footway`, `highway=steps`, `indoor=room`, `entrance=yes`, and per-floor level tags. Before the routing tiles were built, the campus data were hand-edited to add entrance nodes, staircases, and indoor connections; the finished tiles render through MapLibre GL [20].

Routing. Routing uses Valhalla, embedded in the app as an Android Archive. Its pedestrian profile prefers footpaths over roadways, and each route is a list of structured maneuvers, each carrying a turn direction, distance, and type code the speech layer reads out directly.

Accessibility. The app cooperates with the platform screen reader, TalkBack on Android [21], and every control carries a semantic label. Spoken output passes through a priority-queued text-to-speech service, from ordinary prompts at the lowest level up to road-crossing alerts at the highest, so a safety announcement is never cut off by a routine one. English and Arabic both run on the phone’s built-in speech engine and switch from inside the app; no other language is offered.

2.7 Related Work

The assistive-navigation tools in use split into two types. Smartphone-only apps such as BlindSquare and Microsoft Soundscape speak guidance from phone GPS outdoors and, indoors, from WiFi or BLE fingerprinting at 1 to 3 m, too loose for corridor turn cues [9]. Commercial services such as Google Maps and HERE form the other type; they need a constant connection and offer no Arabic pedestrian guidance. Research prototypes using UWB or BLE anchors reach finer indoor accuracy but say nothing about offline operation or Arabic audio [12, 13], and vision-plus-UWB systems hit sub-10 cm only in the lab, at the cost of an onboard camera and its privacy [22, 23].

A 2025 survey of indoor and outdoor localization lands on the same ranking this project reached independently [2]. It puts ultra-wideband at the top for precise indoor positioning, crediting its fine time resolution and low susceptibility to interference. Radio-strength methods such as WiFi RSSI sit lower, since they recover position either by mapping strength to distance or by matching against pre-recorded fingerprints, and the survey reports that

small-scale signal variation degrades both; recent work answers this with adaptive and hybrid models rather than raw strength. Table 2.4 collects representative indoor accuracies from that survey and the wider literature, each value the mean or median error its source reported.

Across the surveyed systems and the wider literature, we found no solution that runs fully offline, guides in both English and Arabic, and pairs sub-metre outdoor GNSS with centimetre-class UWB indoors on self-built hardware. That gap is the opening this project closes: it targets the UWB accuracy class without putting a camera on the user, trading a little best-case error for a wearable that stays simpler, cheaper, and free of a vision sensor’s privacy baggage.

Table 2.4: Indoor positioning techniques and representative accuracy [2].

Technique	Accuracy	Infrastructure	Key Limitation
WiFi RSSI	1–3 m	Existing APs	Multipath; environment-sensitive
WiFi RTT / FTM	0.5–1.5 m	802.11mc-capable APs	Limited to compatible hardware [24]
BLE RSSI	1–3 m	Beacon grid	Same RSSI variance as WiFi [8]
BLE AoA	0.1–0.5 m	Direction-finding anchors	Specialised hardware; higher cost [25]
UWB (adopted)	0.05–0.3 m	UWB anchor network	Anchor deployment cost [12, 13]
RFID	0.5–2 m	Readers + tagged environment	Short read range; passive tags only [26]
Magnetic fingerprint	0.5–2 m	Building survey required	Sensitive to structural changes [11]
Visual / camera	0.1–0.5 m	Camera + visual map	Lighting dependent; privacy
LiDAR	0.05–0.2 m	LiDAR sensor + point-cloud map	Cost; wearability constraints [10]
Acoustic / ultrasound	0.1–0.5 m	Speaker / microphone array	Ambient noise; multipath [27]
ZigBee RSSI	1–5 m	ZigBee mesh network	Low accuracy; sparse deployment
PDR / dead reckoning	1–3% of path	IMU only (no anchors)	Cumulative drift; unbounded error [28]
Vision + UWB fusion	<0.1 m	Camera + UWB anchors	Combined cost; calibration overhead [22, 23]

Chapter 3

System Design and Architecture

3.1 System Overview

Smart Blind System Support is organized as three cooperating tiers. A body-worn sensor hub collects position and motion data, a smartphone application turns that data into a route and spoken guidance, and the user receives directions through audio and vibration. The hub and the phone communicate over a single Bluetooth Low Energy link, and every stage runs without an internet connection, as shown in Fig. 3.1.

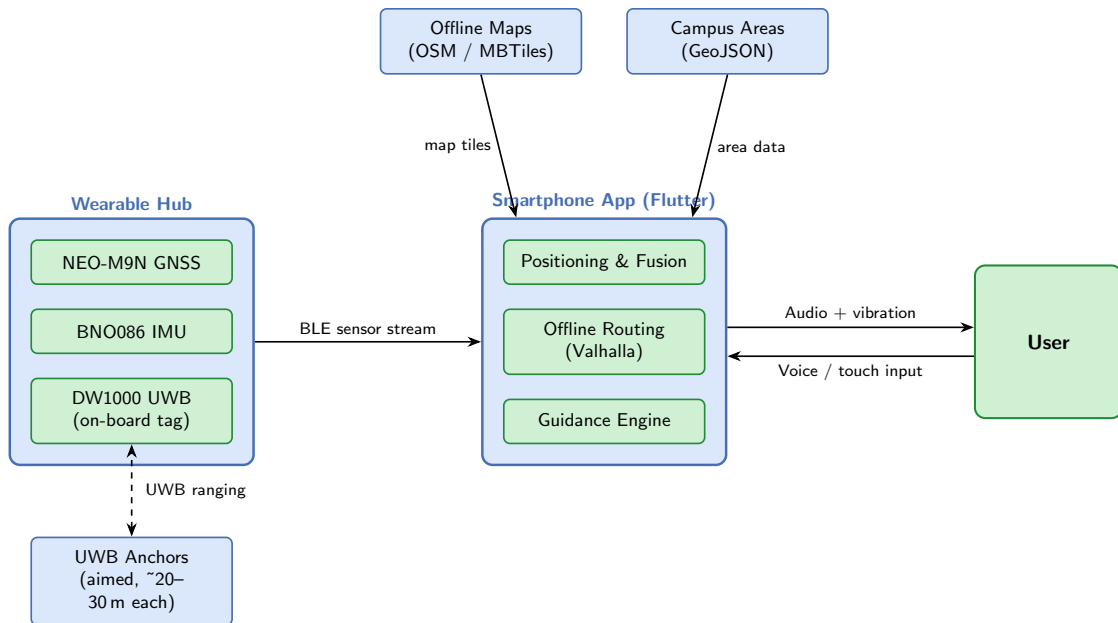


Fig. 3.1: Three-tier system architecture.(Target design.)

Four principles shape the architecture:

- **Offline-first:** all maps, routing, and speech run on the phone, so the system keeps working where there is no mobile signal.
- **Two-domain positioning:** satellite navigation outdoors and ultra-wideband ranging indoors, switched automatically at building entrances.
- **Thin wearable, capable phone:** the hub only senses and forwards data; the phone carries the heavy computation.
- **Accessibility-first:** the user controls the system by voice and receives audio with vibration cues, with no need to read a screen.

The following sections detail each tier: the hardware (Section 3.2), the software (Section 3.3), the indoor–outdoor handoff (Section 3.4), and the end-to-end data flow (Section 3.5).

This chapter describes the *target* architecture — the system as designed. What is implemented and verified in the current build, including where indoor UWB positioning stands, is reported in Chapters 5 and 6.

3.2 Hardware Architecture

The wearable unit is a single ESP32 UWB development board that hosts the NEO-M9N GNSS receiver and BNO086 IMU on separate communication buses and doubles as the system’s UWB ranging tag. A 2S lithium-ion pack, protected by a BMS and regulated to 3.3 V by a buck converter, powers the whole assembly. Fixed UWB anchors installed inside campus buildings are independent nodes; the wearable ranges against them wirelessly for indoor positioning, as shown in Fig. 3.2.

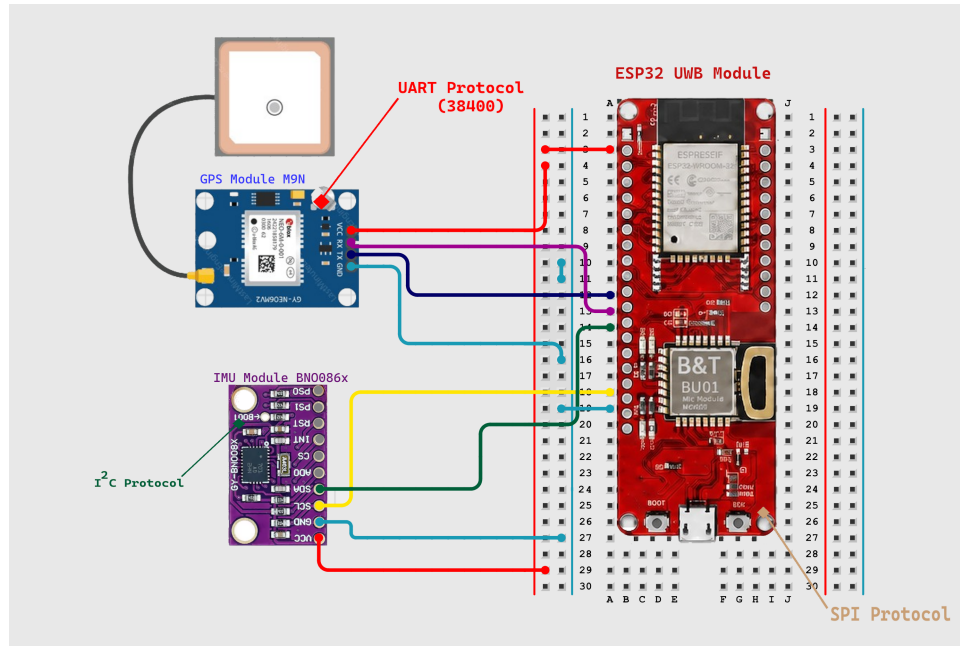


Fig. 3.2: Wearable hardware interconnect.

Table 3.1: Wearable components and interfaces.

Component	Interface	Role
ESP32 UWB (host)	Host MCU + SPI (on-board DW1000)	Main controller, BLE link to phone, on-board UWB tag (see Ch. 4)
NEO-M9N GNSS	UART (GPIO 25/26)	Outdoor satellite positioning
BNO086 IMU	I ² C (GPIO 14/15)	Heading and motion sensing
2S pack + HW-391 BMS + CN3903 buck	Power	Regulates 7.4 V battery to the 3.3 V system rail

The wired-protocol details for UART, I²C, and SPI (timing, voltage levels, and configuration registers) are tabulated in Appendix A.1.

Power chain

- **Cells:** two INR18650-26E lithium cells in series (7.4 V nominal, 8.4 V full).
- **Battery management:** the HW-391 2S BMS is the core of the power board. It controls charging, balances the two cells, and cuts off on over-current, over-voltage, and over-discharge.
- **Charging:** an 8.4 V / 1 A barrel jack feeds the BMS, which manages the charge into both cells in place.
- **Regulation:** a CN3903 buck converter steps the pack down to the 3.3 V rail with far less waste heat than a linear regulator would.

The topology is shown in Fig. 3.3.

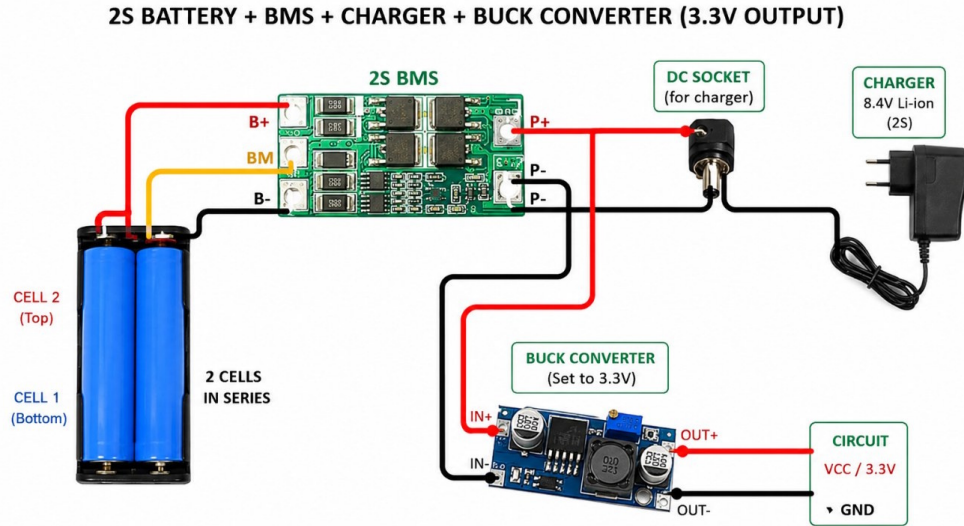


Fig. 3.3: Power subsystem topology.

Each UWB anchor is a separate ESP32-WROOM-32 paired with a DW1000 transceiver, ceiling-mounted on a 3D-printed bracket and aimed into the space it serves. Each anchor covers roughly a half-circle out to 20–30 m in front of it, measured in real testing, and the tag is positioned wherever enough anchor fields overlap; adding anchors extends coverage as a building grows. The pilot used three anchors to trilaterate over the test area. Anchor build and calibration detail appear in the implementation chapter (Chapter 4).

3.3 Software Architecture

Almost all of the system’s logic lives in the Flutter phone application, organized as a stack of layers through which data flows from the sensors down to spoken guidance. The wearable firmware stays deliberately simple: it reads sensors and forwards data, nothing more, as shown in Fig. 3.4.

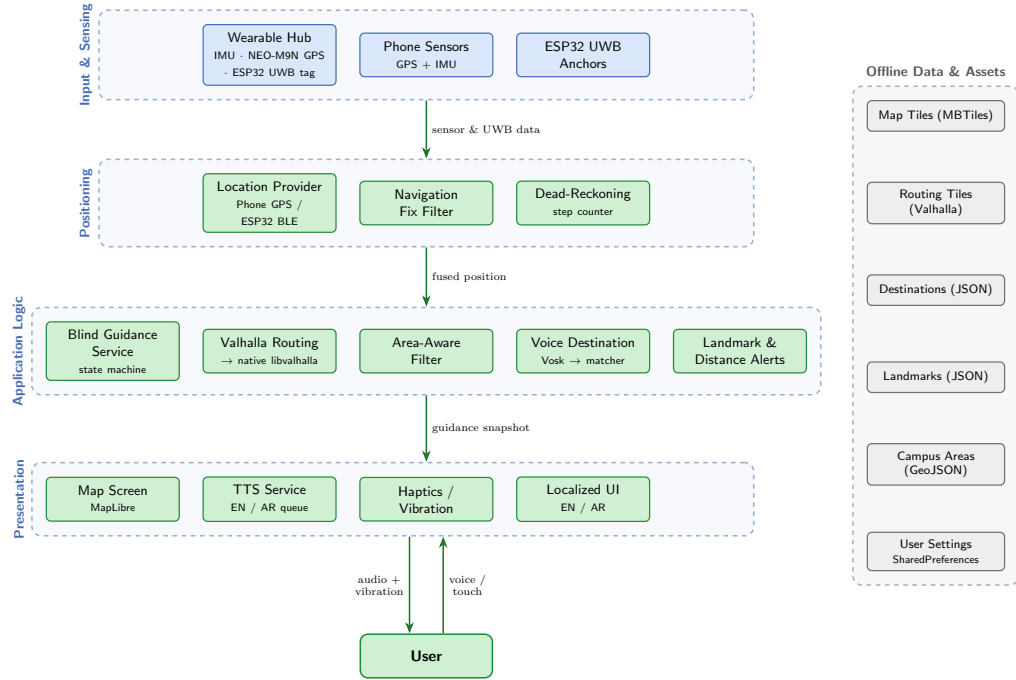


Fig. 3.4: Software architecture.

The layers, in the order data flows through them, are:

- **Input and sensing:** the wearable (GNSS, IMU, and the UWB tag) over Bluetooth Low Energy, the phone's own GPS and compass, and the UWB anchors.
- **Positioning:** a location-provider abstraction over the sources, a fix filter that rejects implausible readings, and a dead-reckoning service that bridges short GNSS gaps, producing one fused position.
- **Application logic:** the blind-guidance state machine, the Valhalla routing service (calling the native engine), the area-aware filter, offline voice-destination search, and landmark and remaining-distance alerts.
- **Presentation:** the MapLibre map screen, the bilingual priority-queued text-to-speech, haptic vibration cues, and the localized (English/Arabic) interface.

All layers draw on bundled offline data: map tiles, routing tiles, the destinations and landmarks catalogues, the campus-areas file, and saved user settings.

Firmware and ranging. The wearable firmware runs as FreeRTOS tasks split across the ESP32's two cores: one core handles GPS and IMU reading, and the other sends BLE notifications, so sensing never blocks transmission. That split keeps the firmware narrow: sense, stream, nothing more. Indoor position is computed by least-squares trilateration: the app resolves the anchor range measurements into an (x, y) coordinate that feeds the

positioning layer alongside the outdoor GNSS/IMU fix. Each anchor runs its own simpler firmware whose only task is to answer ranging requests.

Area-aware guidance. Beside the routing engine the design places a lightweight, read-only spatial layer that holds a compiled map of the campus’s walkable areas, pedestrian streets, obstacles, and stairwells. Valhalla remains the sole authority on the route and any re-route; this layer only judges whether the user is still inside a route-relevant walkable area and, on that basis, lets them move freely across that whole surface toward the target instead of pinning them to the exact route line, suppressing the redundant off-route prompts that pinning would cause. Its as-built behaviour is detailed in Section 4.7.

3.4 Indoor–Outdoor Handoff

Because the system positions one way outdoors (GNSS) and another way indoors (UWB), it needs a controlled switch rather than a hard cut at building entrances. A four-state machine manages that transition, as shown in Fig. 3.5.

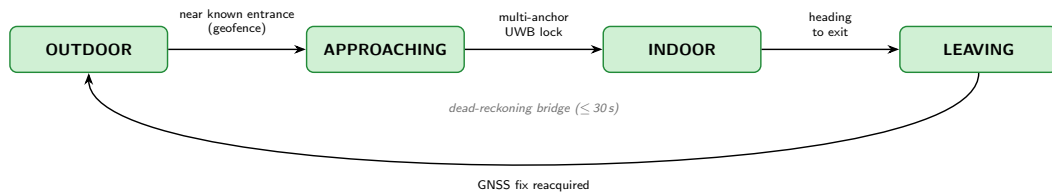


Fig. 3.5: Four-state indoor–outdoor handoff state machine.

The system moves through four states:

- **Outdoor:** GNSS and IMU drive turn-by-turn guidance.
- **Approaching:** near a surveyed entrance, the system watches for a UWB lock while dead-reckoning across the threshold for up to 30 seconds.
- **Indoor:** UWB trilateration takes over, and a spoken cue confirms the switch (for example, “Entering Al-Sadik, switching to indoor guidance”).
- **Leaving:** the system holds in this state until a stable GNSS fix is reacquired, then returns to Outdoor.

Transition triggers:

- **GNSS degradation:** position uncertainty rising past the usable threshold as satellite signals weaken indoors.
- **UWB lock:** a valid range from the building’s UWB anchors.
- **Geofence:** crossing the entry or exit boundary of a building, using entrance coordinates from the GNSS survey.

3.5 Data Flow

Each sensor reading travels from the wearable to the phone and exits as a spoken turn instruction. Outdoors, the ESP32 samples the GNSS and IMU sensors, packs the readings into a compact BLE notification, and the phone receives it several times per second; the app then updates the position estimate, re-evaluates the route, and speaks the next step if anything has changed. Indoors, the UWB ranges reach the app the same way, and the app resolves them by least-squares trilateration into an (x, y) position that feeds the positioning layer, as illustrated in Fig. 3.6.

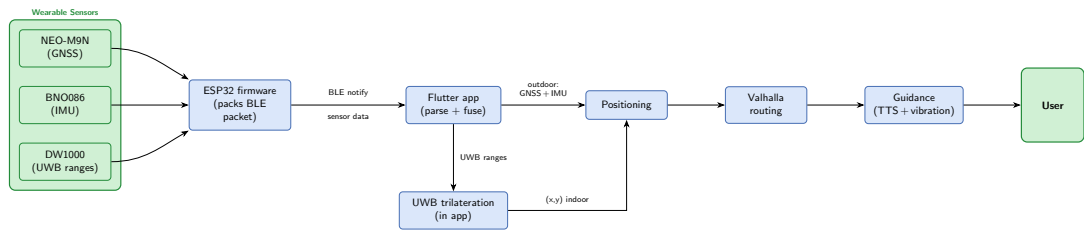


Fig. 3.6: End-to-end data flow.(Target design; the current build runs indoor trilateration on a separate server — see Chapter 5.)

Table 3.2 lists the outdoor BLE packet and the indoor anchor-range telemetry. The outdoor packet carries raw sensor values that the positioning layer fuses on the phone; indoors the tag sends the raw anchor ranges, which the app resolves into a position.

Table 3.2: Outdoor BLE packet and indoor serial telemetry.

Link	Size	Fields
Outdoor: BLE to phone	46 bytes	Timestamp, latitude and longitude ($\times 10^{-7}$ deg), altitude, speed (cm/s), horizontal accuracy (cm), GNSS fix status, orientation quaternion, linear acceleration, and step count
Indoor: anchor ranges $\rightarrow$ app	8 bytes	The raw anchor ranges; the app runs least-squares trilateration to recover the position

Outdoors, the GNSS fix and IMU heading arrive together in the 46-byte packet and feed directly into the fusion and routing layers on the phone. Indoors, the app resolves the raw anchor ranges into an (x, y) coordinate and feeds it into the same positioning layer, so the in-app map dot and guidance work indoors as they do outdoors.

Chapter 4

Implementation

The sections below cover each subsystem in build order. The design and data flow are covered in Chapter 3; the measured accuracy, power, and field-test numbers are reported in Chapter 5.

4.1 Outdoor Positioning

Outdoor positioning pairs a multi-constellation GNSS receiver with a fused IMU heading, adds a dead-reckoning fallback for short signal gaps, and switches to the phone's own sensors automatically when the wearable is not connected.

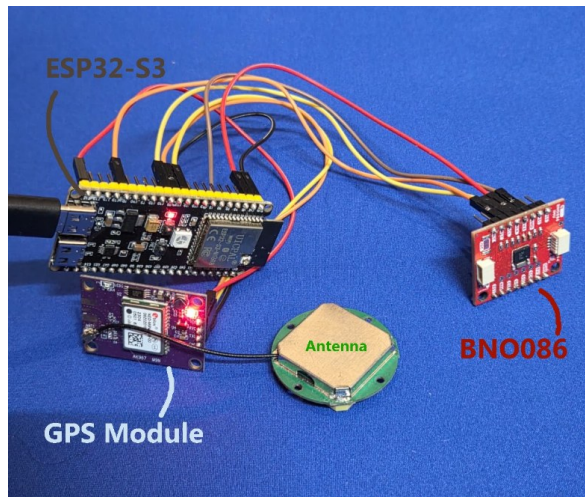


Fig. 4.1: Wearable bench build: the ESP32-S3 with the GNSS and IMU modules.

GNSS receiver.

- **Interface:** the u-blox NEO-M9N talks to the ESP32 over UART at 38400 baud (GPS RX on GPIO 26, TX on GPIO 25) [29]. Firmware enables the UBX binary protocol, so fix fields are read straight from the packet rather than parsed from NMEA strings.
- **Multi-constellation:** GPS, GLONASS, Galileo, and BeiDou are tracked together with SBAS corrections, keeping enough satellites in view where campus buildings block part of the sky.
- **Fix rate and start-up:** Position-Velocity-Time (PVT) solutions are read at 10 Hz, matched to the BLE packet rate so every packet carries a fresh position. A cold start settles in under 30 s; with AssistNow Autonomous the receiver predicts satellite orbits on-device, cutting re-acquisition after a power-off from tens of seconds to a few, still with no network.

- **Fix-quality gating:** a fix is used only if it is a 3D solution from at least five satellites, carries the receiver's own validity flag, and reports horizontal accuracy within 25 m; a 10° elevation mask drops low satellites whose signals reflect off nearby walls. A position that jumps farther than a pedestrian could walk (about 3.5 m/s) is held back unless it repeats, so a brief multipath glitch near a building never reaches the map while genuine movement still does. The firmware also re-applies the full UBX configuration until it takes, rather than silently running on factory defaults after a boot-time race.
- **Firmware task:** `gpsTask`, pinned to core 1, pulls latitude, longitude, speed, and horizontal accuracy from each UBX-NAV-PVT payload into the shared packet buffer.

Heading from the IMU.

- The BNO086 nine-axis IMU sits on I²C (SDA GPIO 14, SCL GPIO 15) [30] and runs in `imuTask` on core 1. A data-ready interrupt line drives each read, so the ESP32 touches the bus only when a sample is waiting instead of polling a busy chip; together with a short bus timeout and a boot sequence that keeps retrying until the sensor answers, this removed the bus lock-ups that used to drop the IMU mid-walk.
- Heading fuses the two orientation sources the chip exposes. The gyroscope gives a smooth, magnet-immune bearing that drifts slowly, while the magnetometer knows true north but spikes near metal. The gyro drives the reported heading and the magnetometer only trims it, capped at 0.25° per update and only while the chip rates its magnetic calibration as reliable. A 90° magnetic disturbance then shifts the heading by about a degree rather than flipping it, while a real turn still follows the body at once.

Dead-reckoning fallback.

- When horizontal accuracy stays above 20 m for two seconds (typical near a building wall), the fix is flagged unreliable and dead reckoning takes over.
- While dead reckoning, heading comes from the BNO086 and each step advances the estimate by a 0.7 m stride. A confidence value starts at 1.0 and decays 0.05 per second, so a longer gap is trusted less.
- The window is hard-capped at 30 s. Once a good fix holds below the error threshold for two seconds, the app reverts to GNSS and resets confidence to full.

Phone-sensor fallback.

- Started without the wearable paired, the app falls back to the Android fused-location provider [31] and runs outdoor guidance on the phone GPS and its motion sensors.

Stream validation. The phone re-checks the wearable’s stream before trusting it. Packets the firmware does not mark valid, impossible or exact-zero coordinates, and positions that fail the same jump/outlier filter used in phone-GPS mode are discarded before they reach the map or the spoken guidance. When the IMU briefly drops out the app holds the last real heading instead of resetting to north, so a sensor hiccup no longer swings the marker.

Both fields travel in the 46-byte outdoor BLE packet (referenced in Chapter 3, Table 3.2); the bench build is shown in Fig. 4.1.

4.2 Testing WiFi RSSI

Before committing to UWB indoors, the team tested whether WiFi received signal strength (RSSI) from the campus access points could place a user inside a building, following the path-loss background in Section 2.3.1. Two things were checked in the Al-Sadik building: whether RSSI could be turned into a usable distance from a single router, and whether the pattern of several routers at a fixed point could identify a location by fingerprinting. This section records how that data was collected; the accuracy outcome that led to dropping RSSI in favour of UWB is reported in Chapter 5 (Section 5.2).

Objective. Collect RSSI readings from the Birzeit campus routers to build a custom equation that approximates distance from signal strength, and to survey the per-router signal at fixed reference points for indoor localization.

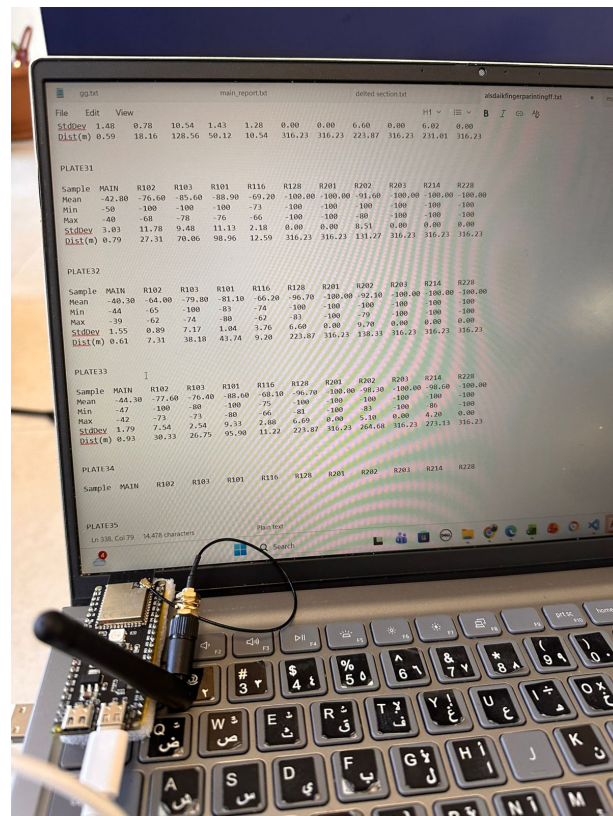


Fig. 4.2: ESP32 scanning the campus WiFi access points during data collection.

Data-collection method.

- An ESP32 scanned the nearby access points under controlled conditions, recording 30 samples at each position and reducing them to a mean, minimum, maximum, and standard deviation per access point.
- For the distance model, the ESP32 was placed directly under the reference router at 2.2 m height, and readings were taken at consecutive 50 cm intervals until the signal was lost. The collected points were then plotted to fit an equation for the RSSI-to-distance relationship.
- The 50 cm step follows the 2.4 GHz wavelength: from (4.1) the wavelength is 0.125 m, and a spacing of $4\lambda = 0.50$ m was chosen for time efficiency while keeping enough resolution.

$$\lambda = \frac{c}{f} = \frac{3 \times 10^8 \text{ m/s}}{2.4 \times 10^9 \text{ Hz}} = 0.125 \text{ m} \quad (4.1)$$

The data-collection setup is shown in Fig. 4.2: the laptop and the ESP32 scanner were carried to each measurement point to log the RSSI values from the WiFi routers in the building.

4.3 Indoor UWB Positioning

The indoor subsystem was built and tested as a standalone setup; this section reports that as-built state. The integrated target design is described in Chapter 3.

Indoor positioning ranges the tag against three ceiling-mounted anchors and solves the ranges into a 2-D position. The tag streams its raw anchor ranges over USB serial to a local server, which runs the trilateration and displays the live position on a floor-plan viewer, a path that is separate from the outdoor BLE link to the phone.

UWB radio configuration.

- The DW1000 transceiver runs on IEEE 802.15.4 [32] in `MODE_LONGDATA_RANGE_LOWPOWER` [33], which sets Channel 5 (≈ 6.5 GHz) and a 16 MHz pulse-repetition frequency by default, trading throughput for range when the tag is ranging to anchors 15–20 m down a corridor.
- The DW1000 is integrated on the Makerfabs ESP32-UWB board, so its SPI wiring to the MCU is fixed on the PCB; the firmware drives it through the open-source `arduino-dw1000` library [34].

Two-Way Ranging.

- Each anchor measures the round-trip time of flight to the tag and converts it to a distance. The tag polls the anchors in turn, producing three fresh ranges at roughly 10 Hz.
- Anchors advertise identifiers $0xA1$, $0xA2$, and $0xA3$; the tag addresses each by identifier, so every range ties to the correct anchor position.

Antenna-delay calibration.

A raw time-of-flight reading carries a fixed offset from the antenna group delay that varies between boards with component tolerances, so each board is calibrated once to cancel it. As shown in Fig. 4.3, the tag and one anchor are set a known 8.0 m apart along the Al-Sadik corridor, where a tape measure gives a clean obstruction-free reference. The antenna-delay constant is then stepped (raising it shortens the reported range by about 20 units per step) until the server reads 8.00 m, then flashed permanently. Table 4.1 lists the final constants.

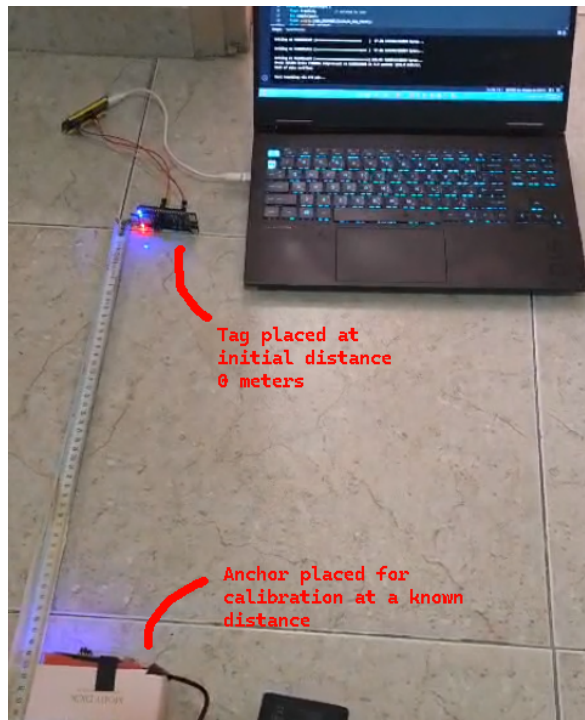


Fig. 4.3: UWB antenna-delay calibration setup.

Table 4.1: Per-board UWB antenna-delay constants.

Board	Antenna delay
Tag (wearable)	16367
Anchor A1	16550
Anchor A2	16521
Anchor A3	16500

Trilateration in the test setup.

- The tag sends its three raw anchor ranges over USB serial to a local Python server (`localhost:5000`), which runs least-squares trilateration to recover an (x, y) position in the Al-Sadik corridor frame. The solve runs on the server, not inside the Flutter app. This is the standalone test arrangement; the target design (Chapter 3) runs the trilateration inside the app.
- The server renders the live tag position over an AutoCAD floor plan of the building, served in a browser. This indoor view is the floor plan itself, not OpenStreetMap; OSM and MapLibre are used only for the outdoor map.
- Ranging gives position but not orientation, so heading still comes from the BNO086. Delivering the solved position to the phone over BLE, so the in-app dot and guidance work indoors as they do outdoors, is left as future work; the BLE link in this build carries only the 46-byte outdoor packet (Table 3.2).

Measured indoor accuracy is reported in Chapter 5. Indoor and outdoor positioning were validated separately; they were not run as one continuous route.

4.4 Routing and Offline Maps

The campus OSM extract, tile build, and MBTiles file all ship inside the APK.

Building the routing graph.

- The campus was hand-edited in OpenStreetMap [35]: the team added every building footprint, named sidewalk and footway, parking area, staircase, and entrance node, giving the router a complete pedestrian graph rather than the sparse off-the-shelf data for the hilltop campus.
- The extract is passed through `osmium sort` and the Valhalla tile builder, producing a compact set of routing tiles (about 10 MB for the whole campus) bundled with the app.

Pedestrian costing.

- Valhalla’s pedestrian costing is weighted to prefer named sidewalks and footways over open road edges, keeping a blind walker on safe paths.
- Surveyed points of interest plus entrance and staircase nodes are tagged into the graph, so a destination resolves to a real door rather than a building centroid.

The pilot covers a single indoor floor; multi-floor routing is not implemented. Extra levels could be added later by authoring OSM `level` tags and pinning each route endpoint to its floor.

Offline map display.

- The outdoor visual map is a pre-rendered MBTiles vector set [36] drawn by MapLibre [37]. Tiles are copied from the app bundle to local storage on first launch, so map and route render with no connectivity.

A close-up of the sidewalk and footpath network the router walks is shown in Fig. 4.4.



Fig. 4.4: Campus sidewalk and footpath network detail.

4.5 Mobile Application

The app ingests a position stream, runs the guidance engine, and speaks instructions through the accessibility layer (Section 4.6).

Provider abstraction.

- Positioning sits behind a single provider interface, so the source can be hot-swapped without touching guidance: the M9N stream over BLE, the Android fused-location provider, and the indoor UWB feed all present the same position object.

- The dead-reckoning service wraps whichever provider is active and fills short GNSS gaps (Section 4.1).

Guidance engine.

- Instructions are staged by distance to the next maneuver: a turn is previewed at 45 m (prepare), called again at 20 m (near turn), with a destination-ahead cue at 18 m and arrival announced at 0.5 m. Maneuvers advance only while moving (above 0.35 m/s), so the guidance does not skip a turn when GPS drifts at a standstill.
- Off-route handling has three selectable sensitivity levels, each with a warning distance, a reroute distance, and a confirmation time before committing to a new route (Table 4.2).
- Heading correction repeats on a timer while the user faces the wrong way, and an on-demand button replays the current instruction.

Table 4.2: Off-route sensitivity levels in the guidance engine.

Level	Warning	Reroute	Confirm time
Very accurate	3.0 m	4.5 m	0.5 s
Mid accurate	6.0 m	8.0 m	1.0 s
Low accurate	10.0 m	14.0 m	2.0 s

Voice search and landmark alerts.

- Destinations can be chosen by voice using the Vosk offline speech recogniser, so search works with no connection.
- A nearby landmark is announced once when the user comes within a configurable proximity radius (20 m by default). Exit hysteresis and a re-announce cooldown keep it from repeating while the user lingers near it. The home screen and voice search are shown in Fig. 4.5 and Fig. 4.6.

4.6 Accessibility and Guidance

Every spoken cue passes through one priority-queued text-to-speech layer, and the whole interface is built to work for a user who never looks at the screen.

Bilingual speech.

- Guidance speaks English or Arabic; the language switches the full spoken vocabulary and the on-screen text. No other language is used.

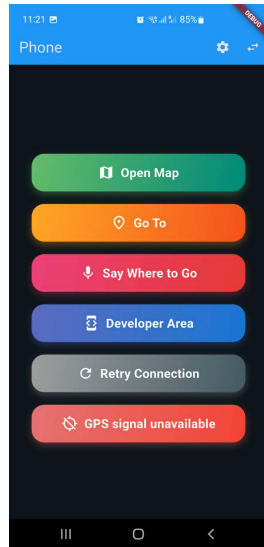


Fig. 4.5: Mobile app home screen.

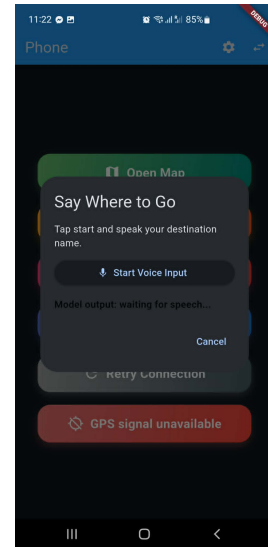


Fig. 4.6: Offline voice destination search.

- The TTS layer is a priority queue, so urgent speech preempts routine speech. Each utterance carries a priority level: general (0), navigation (20), replay (30, an on-demand repeat reusing the navigation channel), warning (40), and alert (100); a higher number jumps ahead, so a hazard alert interrupts a routine distance call.
- Arabic text is segmented from any embedded English so each part is spoken by the right voice, with a transliteration fallback on devices that lack an Arabic engine.

Accessible interface.

- In Arabic mode the layout flips to right-to-left, and a dark high-contrast theme is available. The screens pass through TalkBack without custom work, so users who know those gestures keep them.
- Turns and alerts also fire a phone vibration cue for a non-audio signal in noisy surroundings, and the system speaks a safe street-crossing alert at crossings.

Landmark announcements.

- A nearby-landmark cue gives the name, relative direction, and distance, helping the user build a mental map of the surroundings.
- Twelve landmark types toggle individually: buildings, faculties, libraries, museums, cafeterias, service buildings, gates, bus stations, sport fields, parking, stairs up, and stairs down. The Arabic right-to-left settings screen is shown in Fig. 4.7.

4.7 Area-Aware Command Filtering

A route is a thin line, but the walkable space it runs through — a plaza, a courtyard, a wide path — is usually far wider. The goal of this layer is to let the user move *freely* across that



Fig. 4.7: Arabic (right-to-left) settings screen.

whole allowed surface toward the destination, rather than being pinned to the exact route line. Standard turn-by-turn guidance measures success by distance from the line, so the moment the user steps a little to one side it reads them as off-route and repeats “return to the route.” When the user is in fact still on a valid walking surface and heading the right way, that correction is unwanted: for a blind user it interrupts, contradicts what the cane and feet report, and erodes trust. To grant this freedom of movement inside allowed areas without weakening real warnings, the app runs a second, lightweight spatial layer alongside the Valhalla routing engine.

What the layer knows. The campus is described once, offline, as a map of labelled areas compiled into the app:

- **Allowed surfaces:** walkable areas (plazas, courtyards, wide paths) and pedestrian-accessible streets, stored as polygons with bilingual names.
- **Prohibited areas:** obstacles and stairwells, stored the same way.

Each polygon may carry interior holes (for example a planter inside a plaza), and every area keeps its identifier and an English/Arabic name. The map is treated as advisory: it never overrides the router.

The question it answers. The routing engine stays the single authority on the route, its order, and any re-route; the area layer never invents a path, snaps a position, or changes the route — it reads the route as an opaque line. Each time a recovery prompt is about to be spoken, the layer answers one narrow question: *is the user right now inside an allowed area that the current route actually passes through, at this point along the route?* Answering yes requires all of:

- the live point falls in the *interior* of an allowed polygon (a boundary touch, or a point inside a hole, does not count);
- that polygon is one the active route runs through, and the route’s pass through it brackets the user’s current progress (not far ahead, not behind);
- the position is trustworthy — horizontal accuracy within tolerance and the fix recent.

Any obstacle or stairwell hit is a dominant veto that overrides everything else, and any missing or uncertain input makes the answer default to no.

What it does with the answer. Only on a confident yes is the redundant “return to the route” prompt suppressed: the user is demonstrably still on a legitimate walking surface that the route uses, so the nag adds nothing. If an exact heading-correction cue is already available it is spoken in place of the suppressed prompt; otherwise nothing is spoken. The filter is *fail-open*: on any doubt — weak GPS, a stale fix, an unmapped spot, or a genuine re-route in progress — it keeps the engine’s normal prompt untouched. It can therefore only ever remove a prompt that was already redundant; it cannot invent guidance, and it cannot silence a real off-route or re-route warning. Figure 4.8 summarises the decision.

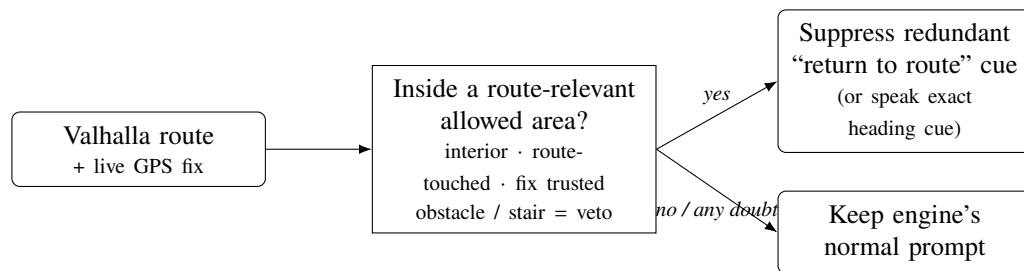


Fig. 4.8: Area-aware filtering of recovery prompts. The router remains the sole routing authority; the layer only suppresses a redundant cue when the user is provably still on a route-relevant walkable surface, and fails open otherwise.

Effect. The result is freedom of movement with a safety net: within a plaza, courtyard, or wide path that the route uses, the user can choose their own line toward the destination and walk it in silence, and a correction is spoken only when they genuinely leave a usable surface. Guidance along corridors and open campus areas becomes quieter and more trustworthy without touching routing accuracy.

4.8 UWB Anchor Deployment

The indoor pilot is the Al-Sadik Building. Three anchors were deployed and tested, placed at corner geometry around the test area to keep dilution of precision low. Each anchor is aimed into the space and covers roughly a half-circle out to 20–30 m in front of it, verified in real testing; the tag is positioned wherever enough anchor fields overlap, and adding anchors extends coverage for a full building.

Anchor firmware.

- Each anchor is a Makerfabs ESP32-WROOM board with a DW1000 radio running responder firmware. It starts as an anchor with its short address (0xA1, 0xA2, or 0xA3) and answers the tag's ranging polls; the per-board antenna delay is the calibrated value from Table 4.1.

Mount and power.

- Each board clips into a 3D-printed PETG bracket (about $73 \times 30 \times 8$ mm, four M3 holes) that leaves the UWB antenna exposed. The bracket attaches to an M10 ball-joint held by a 3D-printed base and a bent metal ceiling clip, letting the anchor be aimed into the room and locked after mounting.
- Each node runs from its own USB power bank, so an anchor needs no ceiling wiring and can be repositioned freely.

The mount mechanism is shown in Fig. 4.9, the clip-and-base assembly in Fig. 4.10, and a deployed anchor in Fig. 4.11.



Fig. 4.9: UWB anchor on the 3D-printed ball-joint mount.



Fig. 4.10: Mount assembly: ceiling clip and ball-joint base.

4.9 Carrier PCB and Power

Carrier PCB.

- A 2-layer EasyEDA design, 82.9×100.8 mm, 1.6 mm thick, with four M3 mounting holes. The ESP32, NEO-M9N, BNO086, and buck converter seat into 2.54 mm female sockets, so any module swaps without soldering.



Fig. 4.11: Deployed UWB anchor on its ceiling mount.

- The 3.3 V rail is decoupled with two 100 μF /16 V bulk capacitors plus 22 μF , 10 μF , and 1 μF values and three 100 nF ceramics at the GPS, IMU, and ESP32 supply pins.
- Protection covers the full input chain: an AO3401A P-MOSFET load switch, an SS14 Schottky for reverse-polarity on the DC jack, an SMBJ9.0A TVS surge clamp, and a JK30 1.6 A polyfuse. A 600 Ω ferrite bead on the GPS supply isolates it from ESP32 RF noise.
- Layout keeps copper clear under the GNSS antenna patch and the ESP32/UWB antennas at the board edges; signal traces ≈ 0.25 mm, power traces ≈ 0.5 mm.
- The five boards were fabricated and received (2026-06-24) but not yet assembled or tested; assembly is the next step.

Power path.

- Two INR18650-26E cells (2600 mAh each) in a 2S pack feed an HW-391 BMS (10 A charge / 20 A discharge) [38], which protects the cells and passes pack voltage to a CN3903 buck converter regulating the 3.3 V rail. A 2S LED gauge shows charge state, and an 8.4 V/1 A CC/CV charger refills through a DC-012 jack.
- A switching buck was chosen over a linear regulator for efficiency: at the wearable's ≈ 310 mA draw, a linear regulator dropping 2S to 3.3 V would burn ≈ 527 mW as heat versus ≈ 90 mW for the buck. Measured runtime is in Chapter 5.
- Table 4.3 compares the candidate sources and the 2S choice; the full power topology

is in Chapter 3 (Fig. 3.3).

Table 4.3: Power-source options considered for the wearable.

Option	Pros	Cons
USB power bank	Simple, off-the-shelf, safe	Bulky, 5 V needs extra regulation, hard to integrate cleanly
1S Li-ion (1 cell)	Small, light, single-cell charging	Lower capacity, shorter runtime, marginal head-room for the buck
2S Li-ion (2 cells)	Higher capacity and run-time, efficient buck step-down, clean BMS balancing	Needs an 8.4 V charger and a 2S BMS

The annotated board layout is shown in Fig. 4.12, the received boards in Fig. 4.13, and the working bench power build in Fig. 4.14.

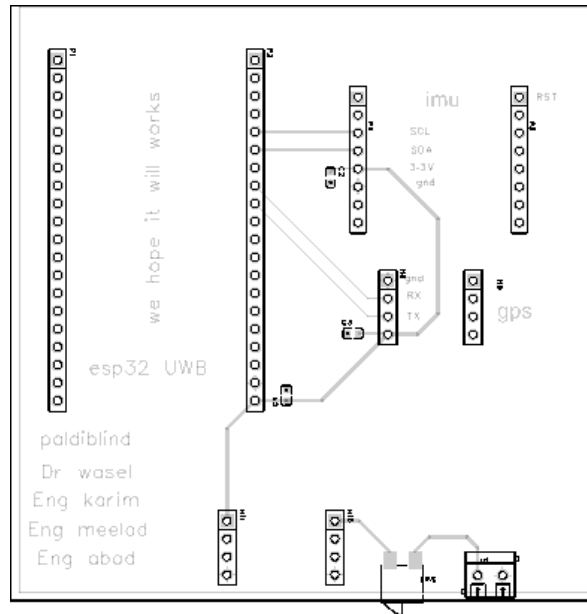


Fig. 4.12: Annotated layout of the 2-layer carrier PCB.

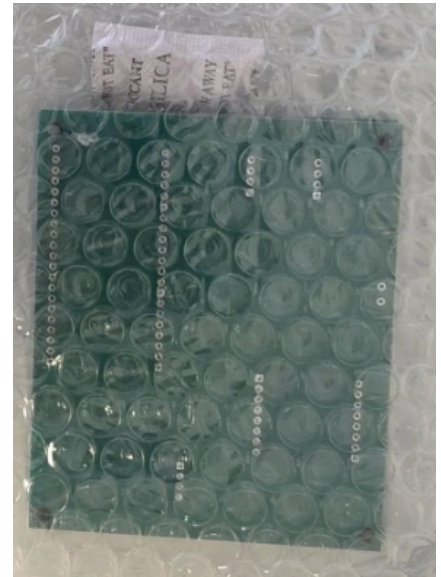


Fig. 4.13: Front and back of the fabricated carrier boards.

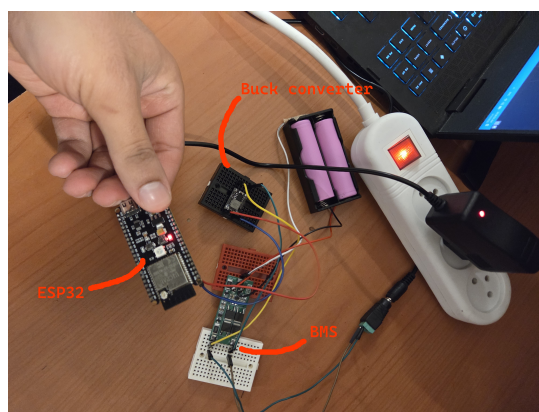


Fig. 4.14: Bench power build.

Chapter 5

Results, Testing and Evaluation

This chapter reports what the system actually measured in the field: outdoor GNSS accuracy against a phone, the WiFi signal-strength study that ruled out RSSI positioning, indoor UWB ranging accuracy, power draw and estimated runtime, and the field tests run with blind students outdoors and with the team indoors. The design behind these subsystems is in Chapter 3 and the as-built detail is in Chapter 4.

5.1 Outdoor GPS Accuracy

The first question was whether a dedicated GNSS receiver really beats the phone a blind user already carries. To answer it the team logged both receivers at the same physical points across campus. At each reference point the tester stood still for about 30 seconds so both fixes could settle, then recorded the phone’s position and the NEO-M9N’s position at the same instant. The M9N coordinates were streamed live from the wearable over BLE. Points were taken at every campus entrance and staircase, the places where a wrong fix does the most harm.

On campus the M9N measured about 0.6 m at best and roughly 1 m on average, well inside its rated 1.5 m CEP [7], while the phone averaged ± 4.2 m across handsets, about four to six times tighter. Table 5.1 lists two paired samples. The horizontal gap between the two receivers at the same point comes out near a metre, which is the difference between landing on the correct door and landing on the wall beside it.

Table 5.1: Paired phone and NEO-M9N fixes at the same points.

Point	Receiver	Latitude, Longitude	Offset
1	Phone GPS	31.9611968, 35.1834773	≈ 1.2 m
	NEO-M9N (BLE)	31.9611868, 35.1834849	
2	Phone GPS	31.9574230, 35.1849020	≈ 1.0 m
	NEO-M9N (BLE)	31.9574180, 35.1849110	

The same comparison is visible in the app itself. As shown in Fig. 5.1, the in-app diagnostic card reports the phone fix at ± 4.2 m next to the M9N fix at ± 0.7 m for the first sample in Table 5.1, so the table and the live card agree.

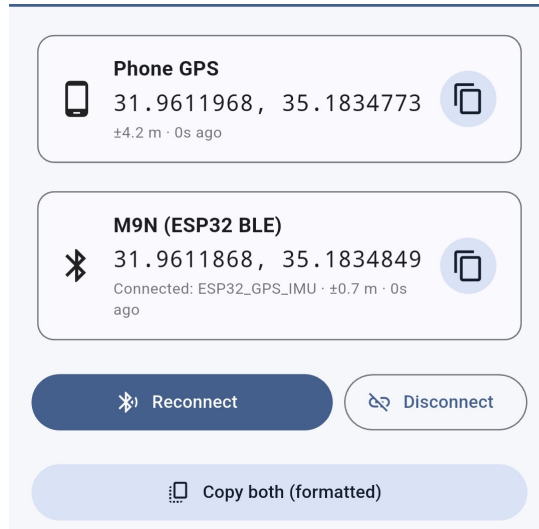


Fig. 5.1: In-app diagnostic card: phone vs. NEO-M9N.

Variation across phones. The phone result depends heavily on the handset. The team’s own test phone, a Samsung Galaxy S25, logged ± 4.2 and ± 6.0 m on campus; the other rows in Table 5.2 are representative values for a span of current and older chipsets, consistent with the observed average rather than raw recorded numbers. Newer dual-band receivers (L1/L5) tend to beat older single-band ones, but none of them reach the M9N’s sub-metre class.

Table 5.2: Phone GPS accuracy by handset.

Phone	Chipset class	Mean horizontal error
Samsung Galaxy S25	Dual-band L1/L5	± 4.2 m
Apple iPhone 14 Pro	Dual-band L1/L5	± 3.1 m
Samsung Galaxy S22	Dual-band L1/L5	± 3.6 m
Samsung Galaxy A52	Single-band L1	± 4.5 m
Xiaomi Redmi Note 11	Single-band L1	± 5.0 m
Mean across handsets		± 4.1 m

Even the best handset (± 3.1 m) is still several times worse than the M9N.

Why phone GPS struggles here. The gap is not just receiver quality. Modern phones lean on assisted GNSS and map-matching, both of which use detailed basemaps and 3D building models to correct a raw fix. That help only exists where the area is richly mapped. As shown in Fig. 5.2, a well-surveyed city in a 3D map tool is dense with accurate building models and high-resolution imagery, so the phone has plenty to match against there.



Fig. 5.2: A richly mapped city in a 3D map tool.

Birzeit is the opposite. As shown in Fig. 5.3, the same tool renders the campus as a flat, low-detail mesh with coarse imagery. On a sparsely mapped hilltop campus the phone gets little correction, so its fix drifts. The NEO-M9N does not rely on any of that: it solves position directly from four satellite constellations, which is why it stays accurate where the basemap is poor. This is also the reason the team surveyed every entrance and staircase itself rather than trusting off-the-shelf map data.



Fig. 5.3: Birzeit campus in the same tool: flat and sparse.

5.2 Evaluation of a Rejected Approach: WiFi RSSI Feasibility

Before settling on Ultra-Wideband (UWB) technology, tests were conducted to determine the feasibility of using Wi-Fi signal strength for indoor positioning based on routers installed in the "Al-Sadik" building. Given that the detected signals fluctuated and lacked stability across various locations, the collected Received Signal Strength Indicator (RSSI) data demonstrated that using these routers as reference points was not the optimal choice for positioning.

Table 5.3: Al-Sadik RSSI Data

	MAIN	R102	R103	R101	R116	R128	R201	R202	R203	R214	R228
Tile 1											
Mean	-51.40	-63.60	-100.00	-86.50	-81.30	-100.00	-100.00	-100.00	-100.00	-100.00	-100.00
Min	-56	-66	-100	-100	-100	-100	-100	-100	-100	-100	-100
Max	-43	-61	-100	-81	-76	-100	-100	-100	-100	-100	-100
StdDev	4.43	1.43	0.00	6.90	6.74	0.00	0.00	0.00	0.00	0.00	0.00
Dist(m)	1.95	7.01	316.23	76.98	44.67	316.23	316.23	316.23	316.23	316.23	316.23
Tile 2											
Mean	-52.30	-71.80	-100.00	-82.90	-93.30	-100.00	-100.00	-100.00	-100.00	-100.00	-100.00
Min	-54	-74	-100	-84	-100	-100	-100	-100	-100	-100	-100
Max	-51	-69	-100	-81	-82	-100	-100	-100	-100	-100	-100
StdDev	0.90	1.40	0.00	0.94	8.22	0.00	0.00	0.00	0.00	0.00	0.00
Dist(m)	2.15	16.53	316.23	52.81	156.84	316.23	316.23	316.23	316.23	316.23	316.23

The two tiles already show the problem: only a handful of routers are heard at all, and the standard deviation on those that are (up to about 8 dBm) shows how much a single router's reading moves at a fixed spot.

The setup was an ESP32 scanning nearby access points while being moved away from a reference point in 0.5 m steps out to about 16 metres. At each step we averaged 30–40 readings and tracked the spread; indoors that spread was the main problem. Fig. 5.4 shows the mean error with its spread at each distance, and Fig. 5.5 shows the raw strength decaying with distance against a fitted path-loss curve.

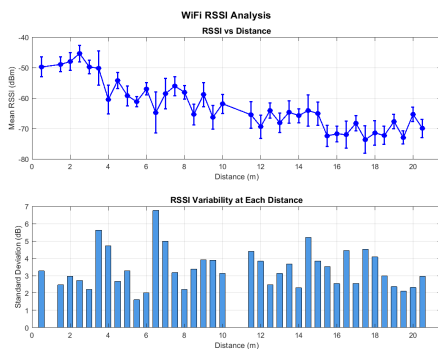


Fig. 5.4: WiFi RSSI positioning error vs. distance.

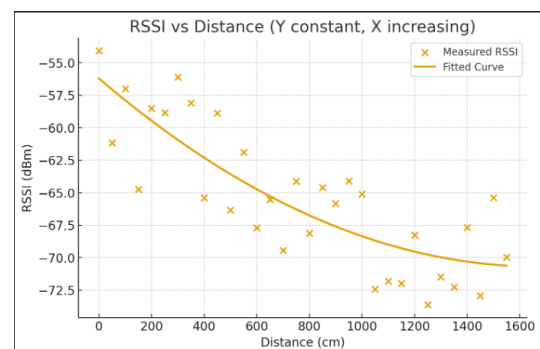


Fig. 5.5: WiFi RSSI decay and fitted path-loss curve.

The results were workable outdoors and poor indoors. A usable fix was obtained about 60% of the time in open outdoor conditions but only around 35% of the time in corridors and stairwells, the exact places a blind user most needs reliable guidance. Within roughly

7 metres of an access point the error sat at 1 to 2 m, but it worsened quickly beyond that, and indoors the reading shifted by several dBm between samples at the same spot, mostly from people walking past or a door opening. An error that size, arriving unpredictably, is not safe to build a turn cue on. At about 35% reliability in corridors, RSSI was not good enough, and the team moved to UWB.

5.3 Indoor UWB Accuracy

Indoor accuracy was checked statically after the per-board calibration described in Chapter 4. The tag was placed at 0.5 m intervals from 1.0 to 5.0 metres from a single anchor along a clear line-of-sight corridor, and 20 readings were averaged at each distance to remove jitter. Across all nine points the absolute error stayed between 5 and 12 cm, averaging 8.3 cm, which sits inside the DW1000 datasheet specification of ± 10 cm [33]. Table 5.4 reports the per-distance result, and Fig. 5.6 plots the measured distance against the true distance with the error at each point.

Table 5.4: UWB ranging error vs. true distance.

True distance (m)	Measured distance (m)	Absolute error (cm)
1.0	1.09	9
1.5	1.57	7
2.0	2.08	8
2.5	2.56	6
3.0	3.11	11
3.5	3.58	8
4.0	4.09	9
4.5	4.55	5
5.0	5.12	12
Average absolute error		8.3

That 8.3 cm is the single-anchor ranging error. Solving three ranges into an (x, y) fix widens it, and anchor placement decides by how much, so the three anchors sit at the corners of the coverage zone to stop small ranging errors from blowing up into large position errors. Fig. 5.7 shows that three-anchor test layout. Walking around the area, the live position settled to about 50 cm, the figure used as the operational indoor accuracy throughout this report.

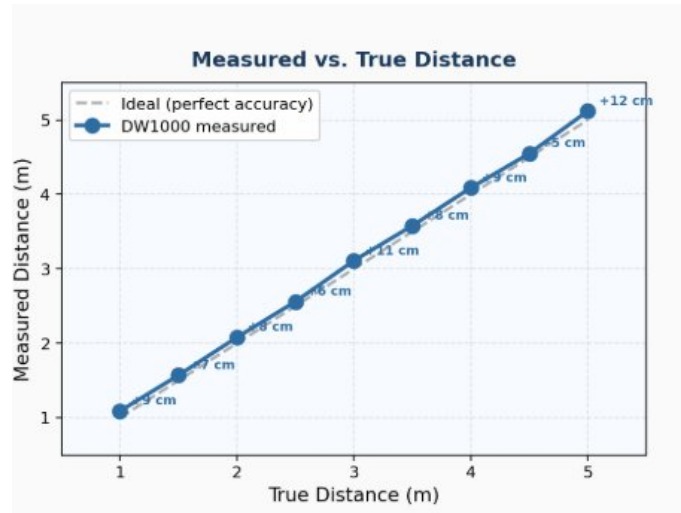


Fig. 5.6: Measured vs. true UWB distance, 1–5 m.

The error grows gracefully with distance, so anchors are spaced 5 to 10 m apart; the radio itself reaches about 100 m outdoors in the open and roughly 30 m down an indoor corridor before the signal degrades.

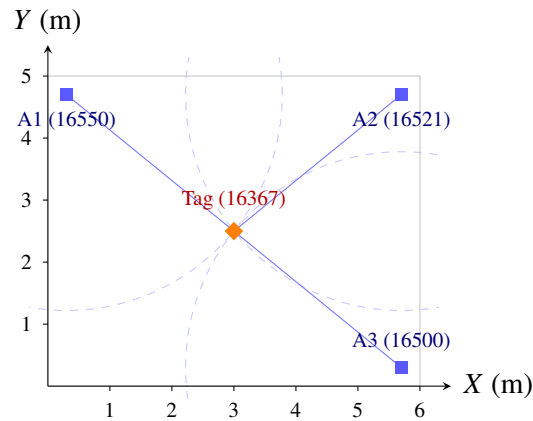


Fig. 5.7: Three-anchor trilateration layout (anchor and tag antenna-delay constants shown).

5.4 Power and Runtime

Each subsystem was measured on its own and then with all of them running at once. Individually the parts sum to about 170 mA; with everything active together the wearable draws about 190 mA at 3.3 V (roughly 627 mW), the extra current coming from concurrent operation. Table 5.5 lists the theoretical and measured draw of each part. The ESP32-S3 (MCU and BLE radio) dominates, while the GPS and IMU are comparatively light.

Table 5.5: Per-subsystem current draw, theoretical vs. measured at 3.3 V. Share is each part's fraction of the 190 mA measured with all subsystems active.

Subsystem	Theoretical (mA)	Practical (mA)	Share
ESP32-S3 (MCU + BLE)	100	103	54%
BNO086 IMU	20	18	9%
NEO-M9N GNSS	50	49	26%
Concurrent overhead	–	20	11%
Total (all active)	170	190	100%

The choice of a switching buck converter over a linear regulator shows up here. Dropping the battery voltage to 3.3 V through a linear regulator at this draw would dissipate around 780 mW as heat, whereas the buck does the same job at about 70 mW, saving roughly 710 mW that would otherwise heat the enclosure. From the 1500 mAh pack used in the build, the theoretical 170 mA draw gives about 8.8 hours of continuous use, and the measured 190 mA draw about 7.9 hours. These runtime figures are estimates derived from the measured draw, not the result of a full battery-endurance run.

5.5 Field Testing

Two separate field tests were run, and they are kept distinct here because they were different in kind: the outdoor test was a real, repeated trial with blind students, while the indoor test was a positioning demonstration run by the team.

Outdoor, with blind students. The outdoor guidance was tested repeatedly with several blind students walking real campus routes. Walking with actual users gave us our most valuable feedback and exposed problems that no bench test would have caught: the rerouting behaviour was the clearest example, where the early version reacted to GPS drift in ways that confused the walker, and watching that happen drove the fix. Their feedback also shaped the adjustable settings shown in this semester's midterm, since no single threshold suited everyone. Settings like the off-route threshold and the landmark toggles exist because users needed guidance they could tune to their own pace. By the later sessions the spoken instructions were timed and worded clearly enough that a student could follow a route entrance to entrance. Fig. 5.8 shows a blind student walking a campus route with the wearable and a white cane.

Indoor, UWB system test. The indoor test was a positioning trial, not a blind-user trial. The team set up three anchors and one tag in one faculty building and walked around while the local server computed and displayed the tag position on the floor plan, holding it to within



Fig. 5.8: Blind student field-testing on a campus route.

about 50 cm. It worked as a positioning demonstration. To be clear about its scope: this was a server-and-viewer test, the position was shown on a computer running the trilateration server, not yet as a live dot inside the phone app, and no blind user took part. The 8.3 cm figure from Section 5.3 is the static per-anchor calibration; the roughly 50 cm here is the moving, three-anchor positioning result. Fig. 5.9 shows the Al-Sadik floor plan used for the test.

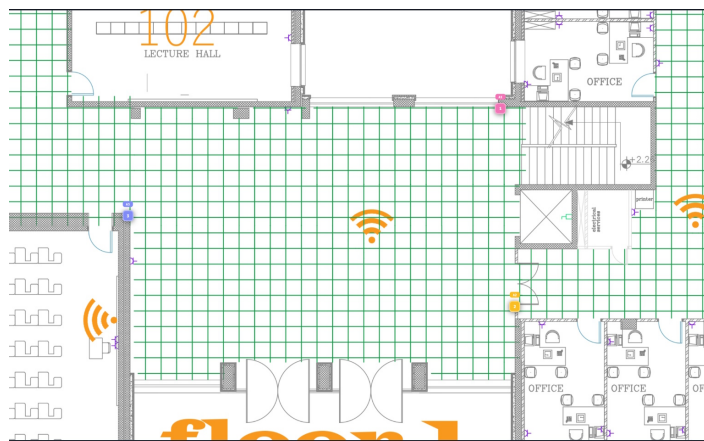


Fig. 5.9: Al-Sadik floor plan used for the UWB test.

5.6 Delivered Feature Set and Limitations

Table 5.6 summarises the working feature set against its tested state. Most of the system is built, integrated, and tested; the honest gaps are named in the same table rather than hidden.

Table 5.6: Delivered feature set and its tested state.

Feature	State
Outdoor GNSS positioning (NEO-M9N)	Working; about 1 m on campus, several times tighter than the phone
IMU heading + dead-reckoning fallback	Working; fills short GNSS gaps
Phone-sensor fallback outdoors	Working; automatic when the wearable is unpaired
Offline routing and maps (Valhalla, MapLibre)	Working; used in every outdoor test
Turn-by-turn guidance + tunable settings	Working; tuned with blind-student feedback
Bilingual EN/AR speech, RTL UI, dark theme	Working
Offline voice destination search	Working; on-device
Indoor UWB positioning (≈ 50 cm)	Server/viewer demo; not yet a live in-app dot
Carrier PCB	Five boards received; not yet assembled or tested
2S battery and charging path	Bench-tested; runtime estimated, not endurance-run

A few limitations follow from these results. Indoor coverage depends on anchor infrastructure installed per floor, so it scales with deployment effort. The operational indoor figure of about 50 cm is looser than the 8.3 cm single-anchor calibration, the expected cost of a moving three-range solution. The carrier PCBs were received but not yet assembled, so all measurements come from the perfboard wearable, and the runtime is estimated from the measured current draw rather than a logged endurance test. None blocks the core result: a fully offline, bilingual aid that positions a blind user to sub-metre accuracy outdoors and about half a metre indoors.

Chapter 6

Conclusions and Future Work

6.1 Conclusions

This project set out to give blind and visually-impaired students a way to move around the Birzeit University campus on their own, without a data connection and in their own language. The result pairs a self-built ESP32 wearable with a Flutter phone application that guides a user turn by turn, with everything it needs held on the device. Outdoors the system met its objectives: the NEO-M9N receiver (rated 1.5 m CEP) held about 0.6 m at best and roughly 1 m on average on campus, several times tighter than the phones we compared, and the BNO086 IMU kept the position alive through short satellite dropouts by dead-reckoning for up to 30 s. When the wearable is not paired, the application falls back to the phone's own location and motion sensors, so a user is never left without guidance. Routing and maps run fully offline from a Valhalla pedestrian graph and MapLibre tiles, both built from an OpenStreetMap extract that we hand-edited to add every campus building, named sidewalk, staircase, and entrance.

The indoor objective was harder, and the honest picture is mixed. A network of DW1000 UWB anchors — each covering a half-circle in front of it out to a tested 20–30 m radius — together with a tag calibrated to an average ranging error of 8.3 cm in static tests, and a local trilateration server tracked the moving tag to within roughly 50 cm while we walked the Al-Sadik faculty building. What it is not yet is a live dot inside the phone application: the solve runs on a local server and viewer, indoor and outdoor were tested separately, and the indoor-to-outdoor handoff between the GNSS and UWB domains is part of the target architecture (Chapter 3) but was not demonstrated end to end. We report this plainly rather than overstate it.

The accessibility layer around the positioning core came together. The application speaks English or Arabic, switchable at any time, with a right-to-left Arabic interface and a priority-queued speech engine so an urgent alert never waits behind a routine announcement; turn cues are reinforced with vibration, road crossings are announced, nearby facilities are called out by name and direction, and an offline voice search picks a destination by speaking. Alongside the routing engine, an area-aware filter reads an authored map of the campus's walkable areas so the user can move freely toward a destination inside the allowed space while redundant or off-area prompts are held back, keeping the spoken guidance calm rather than noisy. The guidance engine was shaped in the field. Testing with several blind students surfaced real bugs, notably in rerouting, and drove the adjustable settings for sidewalk priority, off-route

thresholds, announcement detail, and landmark alerts.

On hardware, the perfboard wearable powers all sensors and streams fused data over Bluetooth Low Energy, the 2S lithium-ion battery and charging path was bench-tested, and the 3D-printed ball-joint anchor mounts were used in the indoor test. The custom two-layer carrier PCB that replaces the jumper-wired prototype was fabricated; five boards arrived but have not yet been assembled or tested.

6.2 Future Work

The first task after submission is to bring up the carrier PCB: populate the five received boards, verify the power and signal nets against the prototype, and run the wearable on them so the dupont-wired build can be retired. That should be paired with a proper battery-endurance measurement, since the quoted runtime is a capacity estimate rather than a measured discharge. The indoor system has the most room to grow: moving the trilateration solve into the Flutter application would turn the positioning into a live in-app dot and let the designed indoor-to-outdoor handoff be exercised end to end at building entrances.

On the firmware side, raising the UWB ranging rate from 10 Hz toward 30 Hz and adding solver-side outlier rejection (discarding readings more than about 20 cm from the running median) would tighten moving accuracy and cut the jitter from non-line-of-sight reflections, while a permanent multi-anchor installation across several buildings would replace the temporary test deployment used for this project. At a larger scale, a structured usability study with a wider cohort of blind students over longer routes would turn the iterative feedback gathered so far into measured evidence. On the software side, extending the authored campus area map to cover the rest of the site and validating the area-aware suppression thresholds with blind users would turn today's draft coverage and informally tuned behaviour into a verified guidance layer. Because the system was built from commodity hardware and offline data, it ports to a new site by surveying OSM edits, building a Valhalla tile extract, and placing UWB anchors, the most direct extension being the other Birzeit faculties already in the campus OSM extract.

Bibliography

- [1] GIS Geography, “Trilateration vs triangulation — how GPS receivers work,” <https://gisgeography.com/trilateration-triangulation-gps/>, 2023, accessed: 2026-06-30.
- [2] M. Stefanoni, I. Kovács, P. Sarcevic, and Á. Odry, “A survey on the main techniques adopted in indoor and outdoor localization,” *Electronics*, vol. 14, no. 10, p. 2069, 2025.
- [3] World Health Organization, “Blindness and vision impairment,” <https://www.who.int/news-room/fact-sheets/detail/blindness-and-visual-impairment>, 2024, accessed: June 2026.
- [4] Market Research Future, “Indoor location market research report — forecast to 2030,” <https://www.marketresearchfuture.com/reports/indoor-location-market-3376>, 2024, accessed: June 2026.
- [5] u-blox AG, *NEO-M9N Integration Manual*, u-blox AG, 2022, document no. UBX-19014285-R07. [Online]. Available: <https://www.u-blox.com/en/product/neo-m9n-module>
- [6] —, *NEO-M8N/NEO-M8M Hardware Integration Manual*, u-blox AG, 2020, document no. UBX-15029985-R04. [Online]. Available: <https://www.u-blox.com/en/product/neo-m8-series>
- [7] —, *NEO-M9N GNSS Module Data Sheet*, u-blox AG, Thalwil, Switzerland, 2023.
- [8] Y. Zhuang, J. Yang, Y. Li, L. Qi, and N. El-Sheimy, “Smartphone-based indoor localization with Bluetooth low energy beacons,” *Sensors*, vol. 16, no. 5, p. 596, 2016.
- [9] R. F. Brena, J. P. García-Vázquez, C. E. Galván-Tejada, D. Muñoz Rodríguez, C. Vargas-Rosales, and J. Fangmeyer, “Evolution of indoor positioning technologies: A survey,” *Journal of Sensors*, vol. 2017, pp. 1–21, 2017, updated survey data from MDPI Sensors 2025 compilation.
- [10] F. Moosmann and C. Stiller, “Velodyne SLAM,” *Proceedings of the IEEE Intelligent Vehicles Symposium (IV)*, pp. 393–398, 2011.
- [11] J. Chung, M. Donahoe, C. Schmandt, I.-J. Kim, P. Razavai, and M. Wiseman, “Indoor location sensing using geo-magnetism,” *Proceedings of the 9th International Conference on Mobile Systems, Applications, and Services (MobiSys)*, pp. 141–154, 2011.

- [12] F. Mazhar, M. G. Khan, and B. Sällberg, “Precise indoor positioning using UWB: A review of methods, algorithms and implementations,” *Wireless Personal Communications*, vol. 97, no. 4, pp. 4467–4491, 2017.
- [13] K. Yu, I. Sharp, and Y. J. Guo, “UWB indoor positioning: Technology, implementation and performance evaluation,” *IEEE Transactions on Instrumentation and Measurement*, vol. 71, pp. 1–12, 2022.
- [14] Qorvo, Inc., *DW1000 IEEE 802.15.4-2011 UWB Transceiver User Manual*, Qorvo, Inc., 2021. [Online]. Available: <https://www.qorvo.com/products/p/DW1000>
- [15] IEEE Standards Association, “IEEE standard for low-rate wireless networks — amendment: Enhanced ultra wideband (UWB) physical layers (PHYs) and associated ranging techniques,” Institute of Electrical and Electronics Engineers, Tech. Rep. IEEE 802.15.4z-2020, 2020. [Online]. Available: <https://standards.ieee.org/ieee/802.15.4z/10230/>
- [16] T. Troger and M. Huber, “arduino-dw1000: An Arduino library for DW1000 ranging and communication,” GitHub repository, 2023. [Online]. Available: <https://github.com/thotro/arduino-dw1000>
- [17] Espressif Systems, *ESP32-S3 Series Datasheet*, Espressif Systems (Shanghai) Co., Ltd., 2023. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32-s3_datasheet_en.pdf
- [18] Bluetooth SIG, “Bluetooth core specification, version 5.0,” Bluetooth Special Interest Group, Tech. Rep., 2016. [Online]. Available: <https://www.bluetooth.com/specifications/specs/core-specification-5-0/>
- [19] OpenStreetMap contributors, “OpenStreetMap: Free and editable map of the world,” <https://www.openstreetmap.org>, 2024, accessed: June 2026.
- [20] MapLibre Contributors, “MapLibre GL: Open-source map rendering engine,” <https://maplibre.org>, 2024, accessed: June 2026.
- [21] Google LLC, “TalkBack screen reader for Android: Accessibility help,” <https://support.google.com/accessibility/android/answer/6283677>, 2024, accessed: June 2026.
- [22] C. Liu, C. Zhao, and J. Ding, “Indoor positioning system based on UWB and vision sensor fusion,” *IEEE Sensors Journal*, vol. 20, no. 20, pp. 12 263–12 273, 2020.
- [23] X. Li, J. Wang, and C. Liu, “A tightly-coupled UWB/IMU fusion method for indoor localization,” *Remote Sensing*, vol. 14, no. 7, p. 1715, 2022.

- [24] M. Ibrahim, H. Liu, M. Jawahar, V. Meenakshisundaram, M. Gruteser, R. Howard, B. Yu, and F. Bai, “Verification: Accuracy evaluation of WiFi fine time measurements on an open platform,” *Proceedings of the 24th Annual International Conference on Mobile Computing and Networking (MobiCom)*, pp. 417–421, 2018.
- [25] M. Cominelli, P. Patras, and F. Gringoli, “Dead on arrival: An empirical study of the Bluetooth 5.1 positioning system,” *Proceedings of the 13th International Workshop on Wireless Network Testbeds, Experimental Evaluation & Characterization (WiNTECH)*, pp. 13–20, 2019.
- [26] L. M. Ni, Y. Liu, Y. C. Lau, and A. P. Patil, “LANDMARC: Indoor location sensing using active RFID,” *Wireless Networks*, vol. 10, no. 6, pp. 701–710, 2004.
- [27] M. Hazas and A. Ward, “A high performance privacy-oriented location system,” *Proceedings of the First IEEE International Conference on Pervasive Computing and Communications (PerCom)*, pp. 216–223, 2003.
- [28] A. R. Jimenez, F. Seco, C. Prieto, and J. Guevara, “A comparison of pedestrian dead-reckoning algorithms using a low-cost MEMS IMU,” *Proceedings of the IEEE International Symposium on Intelligent Signal Processing (WISP)*, pp. 37–42, 2009.
- [29] u-blox AG, “NEO-M9N Standard Precision GNSS Module — Data Sheet,” <https://www.u-blox.com/en/product/neo-m9n-module>, 2024, multi-constellation GNSS receiver (GPS/GLONASS/Galileo/BeiDou). Accessed: 2026-05-18.
- [30] CEVA / Hillcrest Labs, “BNO086 9-Axis Smart Sensor with Sensor-Hub Data Sheet,” <https://www.ceva-dsp.com/product/bno086/>, 2024, inertial measurement unit used for heading and dead reckoning. Accessed: 2026-05-18.
- [31] Google Developers, “Fused Location Provider API,” <https://developers.google.com/location-context/fused-location-provider>, accessed: February 2026.
- [32] IEEE Standards Association, “IEEE 802.15.4z-2020 — Enhanced Ultra Wideband (UWB) Physical Layers (PHYs),” <https://standards.ieee.org/ieee/802.15.4z/7029/>, 2020, the standard the DW1000 ranging protocol implements. Accessed: 2026-05-18.
- [33] Qorvo (formerly Decawave), “DW1000 IEEE 802.15.4-2011 UWB Transceiver — Data Sheet,” <https://www.qorvo.com/products/p/DW1000>, 2022, centimetre-level UWB ranging IC used in all PaldiBlind anchors and tags. Accessed: 2026-05-18.
- [34] T. Trojer, “arduino-dw1000 — Arduino driver library for the Decawave DW1000 UWB transceiver,” <https://github.com/thotro/arduino-dw1000>, 2024, open-source SPI driver used as the low-level layer for all PaldiBlind UWB nodes. Accessed: 2026-05-18.

- [35] OpenStreetMap Foundation, “OpenStreetMap,” <https://www.openstreetmap.org/>, 2025, source of the underlying campus map data. Accessed: 2026-05-18.
- [36] MapBox, “MBTiles Specification 1.3,” <https://github.com/mapbox/mbtiles-spec>, 2024, the container format used to ship offline campus map tiles inside the app. Accessed: 2026-05-18.
- [37] MapLibre Organization, “MapLibre Native — Interactive Vector Map Renderer,” <https://maplibre.org/>, 2025, vector-tile renderer powering the offline campus map in the PaldiBlind Flutter app. Accessed: 2026-05-18.
- [38] HW Power Module Vendor, “HW-391 2S Li-ion BMS Protection Board (20A discharge, 10A charge),” <https://www.electronicspices.com/HW-391>, 2024, protection-only BMS used in the discrete charging-circuit build. Accessed: 2026-05-18.
- [39] R. Mischianti, “I2C, SPI and UART Compared: Understanding Communication Protocols,” <https://mischianti.org/i2c-spi-and-uart-compared/>, accessed: November 2025.
- [40] u-blox AG, *NEO-M8N GNSS Module Data Sheet*, u-blox AG, Thalwil, Switzerland, 2016.

Appendix A

Technical Specifications

This appendix collects the hardware-level details that support the design and implementation chapters: the wearable’s pin assignments, a comparison of the three serial buses used on the board, and the reasoning behind the chosen GNSS and UWB chips.

A.1 Wearable Module Pinout

The wearable is built around a single ESP32 host. Three sensors hang off it on three different buses: the NEO-M9N GNSS receiver on a hardware UART, the BNO086 inertial measurement unit on I²C, and the DW1000 Ultra-Wideband transceiver on SPI. Table A.1 lists the exact GPIO assignment taken from the wearable firmware.

Table A.1: ESP32 wearable pin assignment.

Module	Bus	Signal	ESP32 GPIO
NEO-M9N (GNSS)	UART	RX (host ← GPS TX)	26
		TX (host → GPS RX)	25
		Baud rate	38400, 8N1
BNO086 (IMU)	I ² C	SDA	14
		SCL	15
		RST	13
		7-bit address	0x4B
DW1000 (UWB)	SPI	SCK	18
		MISO	19
		MOSI	23
		SS (chip select)	4
		IRQ	34
		RST	27

All values were read directly from the wearable firmware and match the wiring of the assembled board. The GNSS link runs at 38400 baud rather than the more common 9600

so the receiver can stream 5 Hz position-velocity-time messages without back-pressure. The IMU reset line (GPIO 13) lets the host pulse the BNO086 on a warm reboot to clear a stuck I²C state.

A.2 Serial Bus Comparison

The three sensors deliberately use three different buses, each suited to the kind of data it carries. Table A.2 contrasts the buses on the points that mattered for this design; the qualitative comparison is referenced from §2.5 and §3.2. The figures are typical values for embedded use rather than absolute limits [39].

Table A.2: Serial bus comparison.

	UART	I ² C	SPI
Data wires	2 (TX, RX)	2 (SDA, SCL)	3 shared (SCK, MOSI, MISO) + 1 SS per device
Clocking	Asynchronous (agreed baud rate)	Synchronous (shared clock)	Synchronous (shared clock)
Typical speed	up to ~1 Mbit/s	100 / 400 kHz (up to 3.4 MHz)	tens of MHz
Devices per bus	Point-to-point (one peer)	Up to 127, addressed in software	Several, one chip-select line each
Used in our system	NEO-M9N GNSS	BNO086 IMU	DW1000 UWB

UART suits the GNSS receiver because it streams a steady, one-directional flow of fixes and needs no clock line. I²C keeps the IMU wiring to two pins, which is enough for the modest sample rate of fused orientation data. SPI gives the DW1000 the high, low-latency clock it needs to exchange ranging frames quickly, and its dedicated chip-select fits a single fast peripheral cleanly.

A.3 GNSS Receiver Selection: NEO-M9N versus NEO-M8N

An earlier prototype used the u-blox NEO-M8N. It was replaced by the NEO-M9N because the newer module tracks more constellations at once and reports tighter fixes. Table A.3 summarises the difference; values are paraphrased from the manufacturer datasheets [7, 40].

Table A.3: NEO-M9N versus NEO-M8N.

	NEO-M8N	NEO-M9N
Concurrent constellations	Up to 3	4 (GPS, GLONASS, Galileo, BeiDou) + SBAS, QZSS
Max update rate	Up to 10 Hz (lower with multi-GNSS)	Up to 25 Hz
Horizontal accuracy class	~2.5 m CEP	~1.5 m CEP
On-campus accuracy (measured)	coarser, ~1.2–2.5 m	~0.6 m at best, ~1 m on average

Tracking four systems concurrently means more satellites are visible between buildings and on the hilltop campus, which both lowers the dilution of precision and improves multipath rejection. The module is rated at 1.5 m CEP [7]; on open campus paths we measured about 0.6 m at best and roughly 1 m on average, several times better than a typical smartphone, which is why it became the primary outdoor source.

A.4 UWB Transceiver Selection: DW1000 versus DW3000

The indoor positioning uses the Decawave/Qorvo DW1000. Its successor, the DW3000, is newer and more power-efficient, but the DW1000 was the better fit for this project. Table A.4 lists the main differences.

Table A.4: DW1000 versus DW3000.

	DW1000	DW3000
UWB channels	1–5 and 7 (3.5–6.5 GHz)	5 and 9 (6.5 and 8 GHz)
Standard	IEEE 802.15.4-2011	IEEE 802.15.4z [32]
Power	Higher transmit/receive current	Lower current, deeper sleep
Ecosystem	Mature open Arduino library [34]	Newer, fewer ready-made libraries

The DW1000 was chosen for its mature, openly available Arduino driver and because the Makerfabs ESP32 UWB boards carry it as standard [33]. Its ranging accuracy already meets the sub-decimetre target, so the DW3000’s lower power draw did not justify rebuilding the firmware around a less-supported part.

Appendix B

Demonstration Videos

Short video recordings of the system in use are too large to embed in this document, so they are hosted online. They show the outdoor turn-by-turn guidance on campus, the bilingual spoken instructions, the offline map and routing, and the indoor UWB positioning demonstration. The clips can be reached through the link below or by scanning the QR code.



<https://drive.google.com/drive/folders/1LG5DV4ixvopgoFRLE0iBXhppWSqk7sRd>